

Student Name
Karan Sah

Student Number
2329860

Portfolio Introduction

Workshop Activities 50% Weighting

Mini Project 50% Weighting

This completed portfolio will need submitting to Canvas by the due date.

Questions please email

Dr Sarah Slater

s.i.slater@wlv.ac.uk

Portfolio

Contents

Workbook 1	4
Activity 1.1: Actual voltage across 5V breadboard pins.	4
Activity 1.2: Actual voltage across 3.3V breadboard pins.....	5
Activity 1.3: Potential Divider Calculations	6
Activity 1.4: 3V Calculations from either the 5V supply or 3.3V supply	7
Activity 1.5: Voltage Divider circuit readings from Breadboard circuit.	7
Activity 1.6: LED Circuits	8
Activity 1.7: Current Measurement	9
Activity 1.8: Fritzing for 4 switches & LEDs	10
Activity 1.9: Fritzing for Number 0-7	11
Workbook 2	20
Activity 2.1: LED Flashing to show decimal number 63 as binary.	20
Activity 2.2: 4 LED's for counting up in binary from 0 to 15.	21
Activity 2.3: Traffic Lights.....	24
Workbook 3	27
Activity 3.1: Circuit Diagram of Button & LED	27
Activity 3.2: 3 Switches & Led	29
Activity 3.3: 8 Buttons & LEDs (SWITCH STATEMENTS).....	32
Workbook 4	36
Activity 4.1: Serial Port.....	36
Activity 4.2: Serial Port binary to decimal.....	40
Activity 4.3: Calibrating Analogue Information	43
Activity 4.4: Temperature Sensor & Serial Port	47
Workbook 5	50
Activity 5.1: RGB Led and switches.....	50
Activity 5.2: Distance Sensor	53
Activity 5.3: 1602 LCD Display	56
Workbook 6	61
Activity 6.1: PWM	61

Workbook 7	63
Activity 7.1: 2 Arduinos – using Digital Pins	63
Activity 7.2: 2 Arduinos – using Serial I/O	68
Workbook 8	72
Activity 8.1: Stepper Motor Circuit Diagram	72
Activity 8.2: 2 Stepper Motors	75
Workbook 9	76
Activity 9.1: Windscreen Wiper Code using Servos & Temperature Sensor	76
Individual Project (50%)	80
Rationale	80
Timescales	81
Equipment	81
The Project	81
Step 1 produce a detailed description of your project.	81
Step 2 Circuit Diagram & Fritzing Schematic	81
Step 3 A Program	81
Step 4 Testing	81
Step 5 Conclusions	82
Layout	82
Marking	82
All sections carry equal marks	82

If you prefer, you may use Tinkercad to show a component layout, rather than a circuit Diagram in Fritzing or other circuit design software, though a circuit diagram is more useful as this is what you would most likely see if you were working on embedded systems.

Workbook 1

Activity 1.1: Actual voltage across 5V breadboard pins.

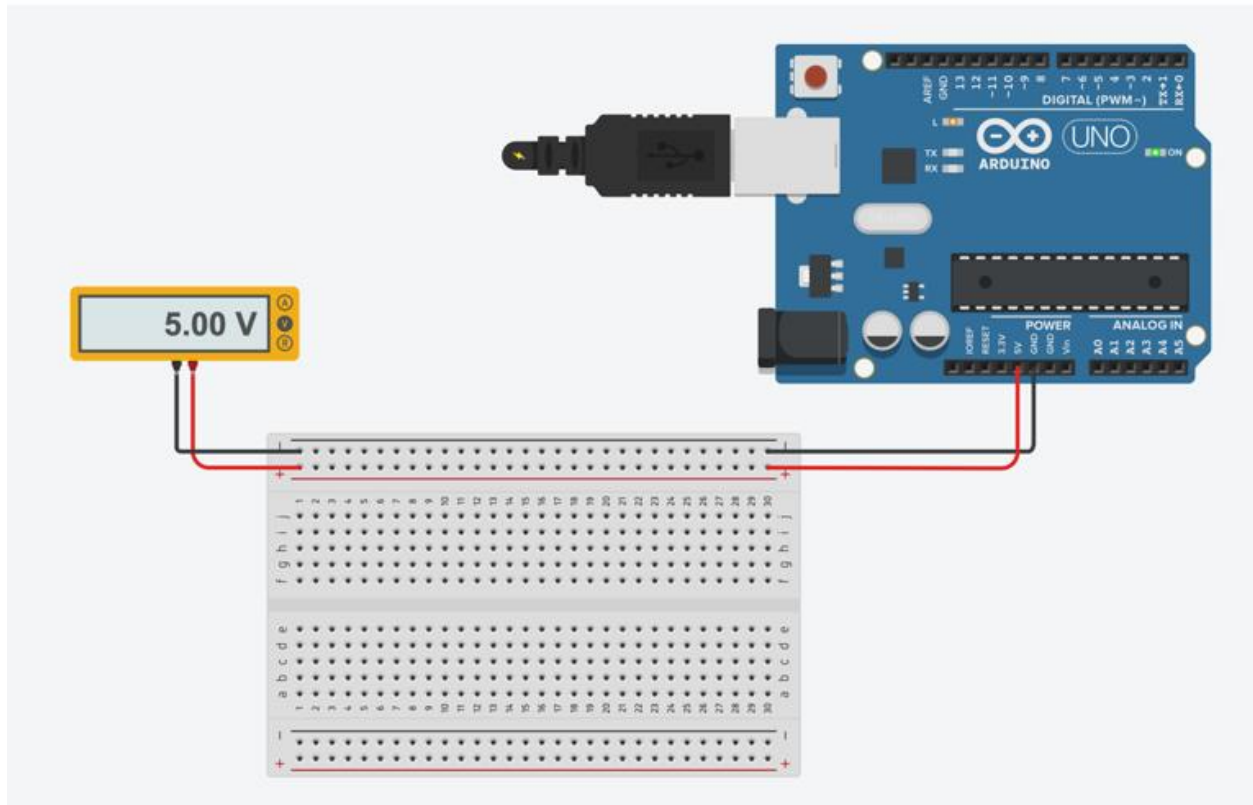


Figure 1 Actual Voltage Across 5V Breadboard Pins (Karan Sah)

Activity 1.2: Actual voltage across 3.3V breadboard pins.

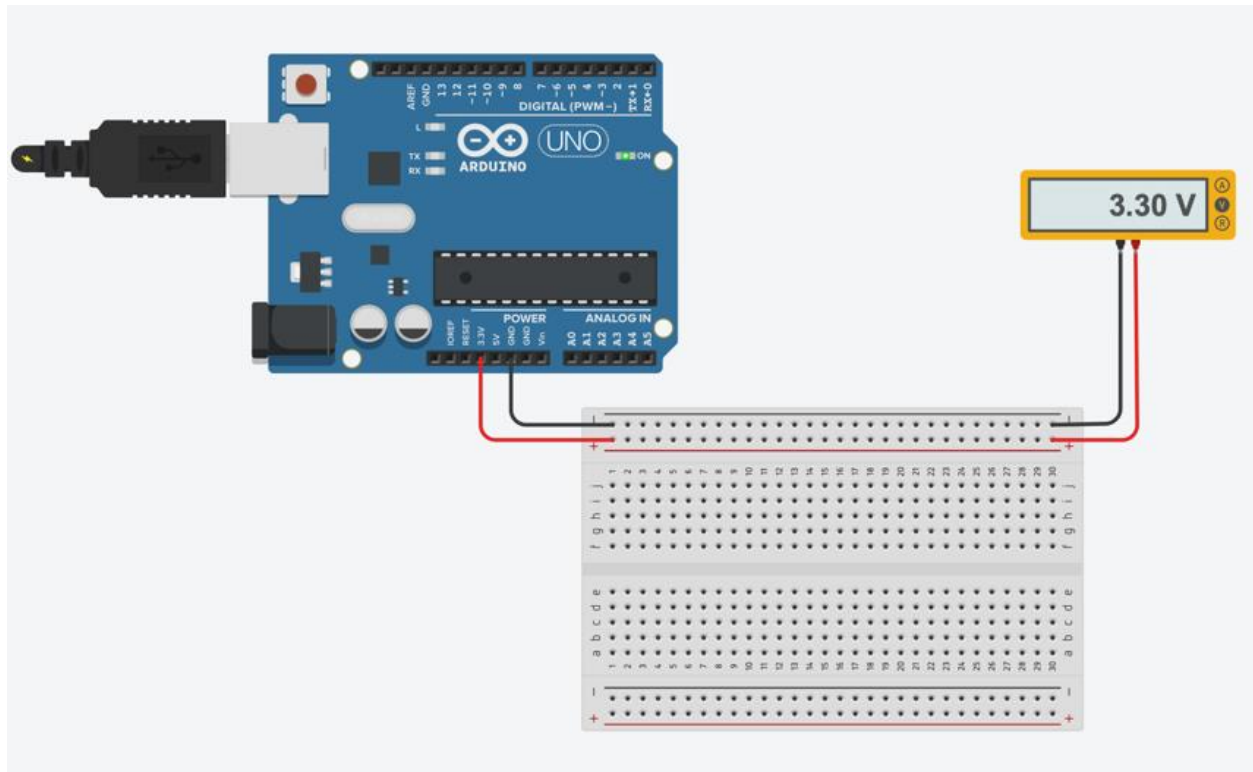


Figure 2 Actual Voltage Across 3.3V Breadboard pins (Karan Sah)

Explain in around 100 words why you think the value read by a multi meter on a circuit, may be different to a simulator value such as Tinkercad.

The reason that the value on a simulator value such as Tinkercad is different to that of on a multi meter on a circuit is that Tinkercad displays ideal value of voltage, that is simple factors are considered and not the real world situations.

In tinkercad, by simply using the factors of current, resistor and voltage the voltage is shown. But different value is shown during real life is due to the different environmental, internal and external factors. Like temperature and humidity can alter the values. Similarly, the type of wire used can also effect the value like, thickness of the wire, length of the wire, heat factors. Internal resistance and the metal used for wire are also among the factors that affect the final value.

If the read value is 4.84V on a 5V supply, what would be a sensible tolerance to quote, explain your answer.

$$\text{Tolerance} = [(5 - 4.84) / 5] * 100 = 3.2\%$$

Hence, the sensible tolerance to quote will be $\pm 3.2\%$ due to the impact of different factors.

The measured value in multi meter can be either plus by 3.2% or down by 3.2%. However, it still can be regarded as accurate.

Activity 1.3: Potential Divider Calculations

Show the working on how you achieved 2.5V

□

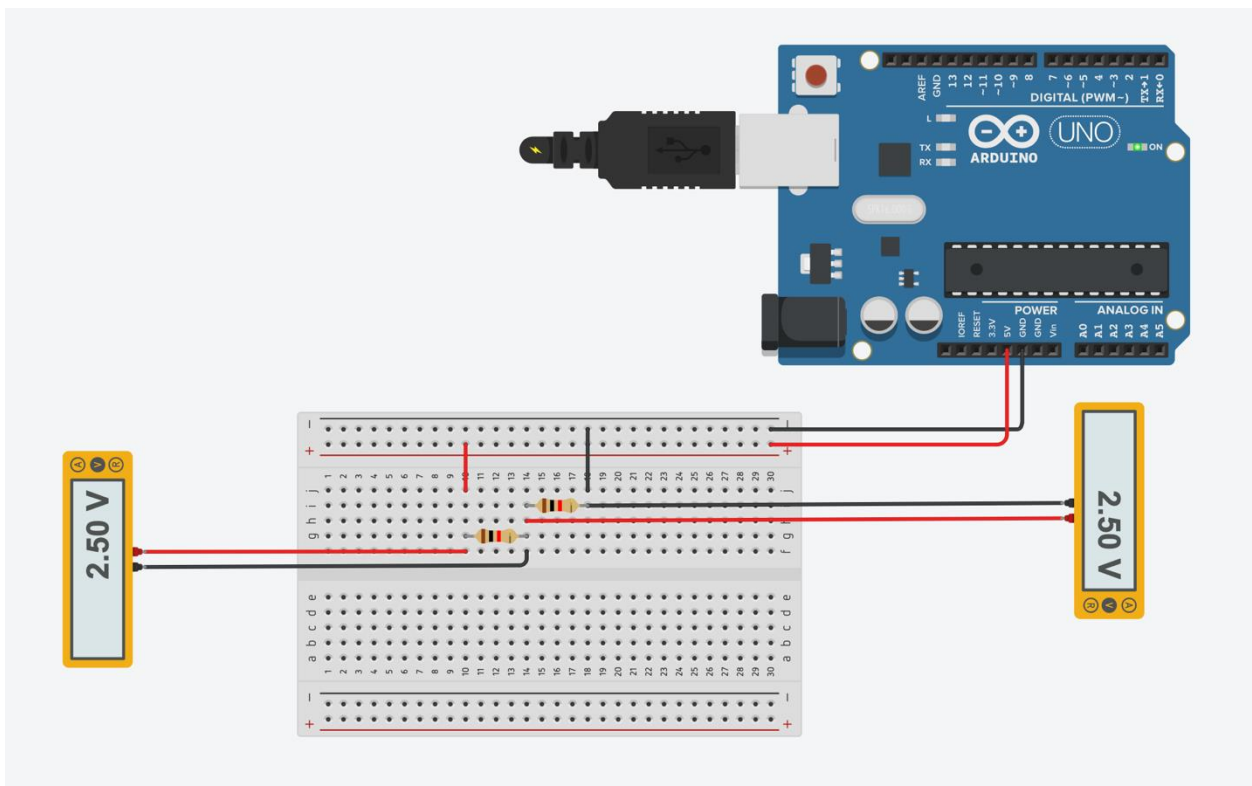


Figure 3 Achieving 2.5 V (Karan Sah)

Activity 1.4: 3V Calculations from either the 5V supply or 3.3V supply

Here,

$$V_s=5V$$

$$V_1=2V$$

$$V_2=3V$$

$$R_1=220$$

$$R_2=?$$

$$R_t=?$$

We know,

$$V_1=I \cdot R_1$$

$$I=2/220=1/110$$

Current, I is same in series.

Therefore,

$$R_2=V_2/I$$

$$=3/(1/110)=330V$$

Activity 1.5: Voltage Divider circuit readings from Breadboard circuit.

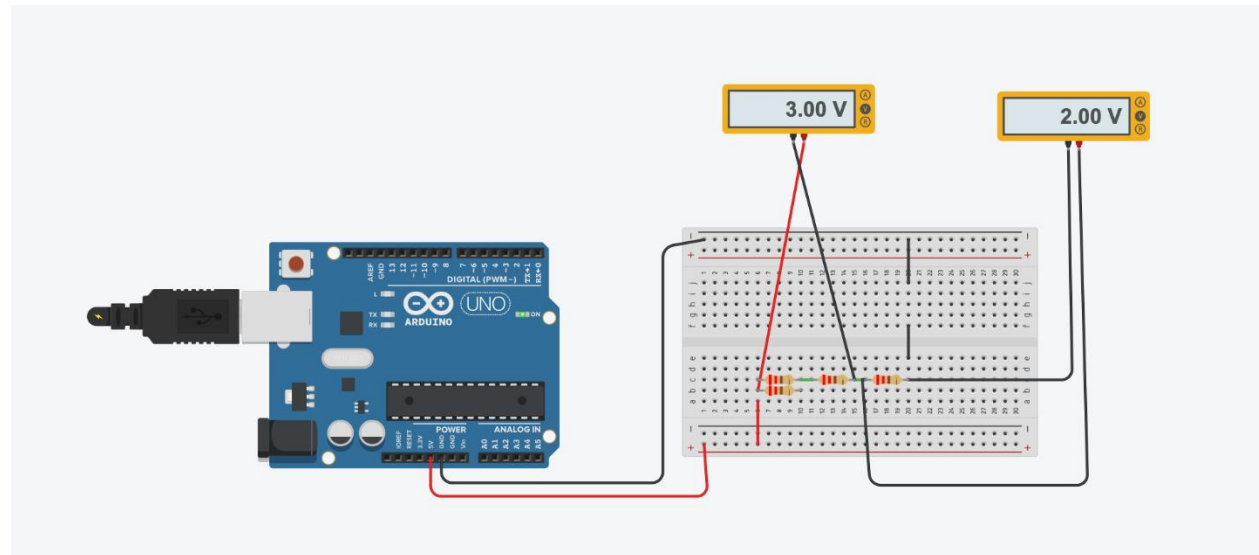


Figure 4 Voltage Divider Circuit Reading From Breadboard Circuit into 3V and 2V (Karan Sah)

Activity 1.6: LED Circuits

Each resistor Value

220 Ohm

220 Ohm

Total resistance Calculation

$$1/R_t = 1/220 + 1/220$$

$$R_t = 110 \text{ ohm}$$

Measured Resistance

108.9 ohm

If measured resistance is not the same, why not? If you simulated this, why might the real value be different.

The measured resistance is not the same due to several external and internal factors. And the measured resistance is not the exact value of the simulation but a value that is very close in approximation to the value.

The real value might be different due to the factors like tolerance of the resistor, ambient temperature, thickness and area of the wire, length of the wire used, the resistivity of the conductors used in the resistor, temperature of the wire.

Activity 1.7: Current Measurement

Calculation of current flowing into LED

0.045A

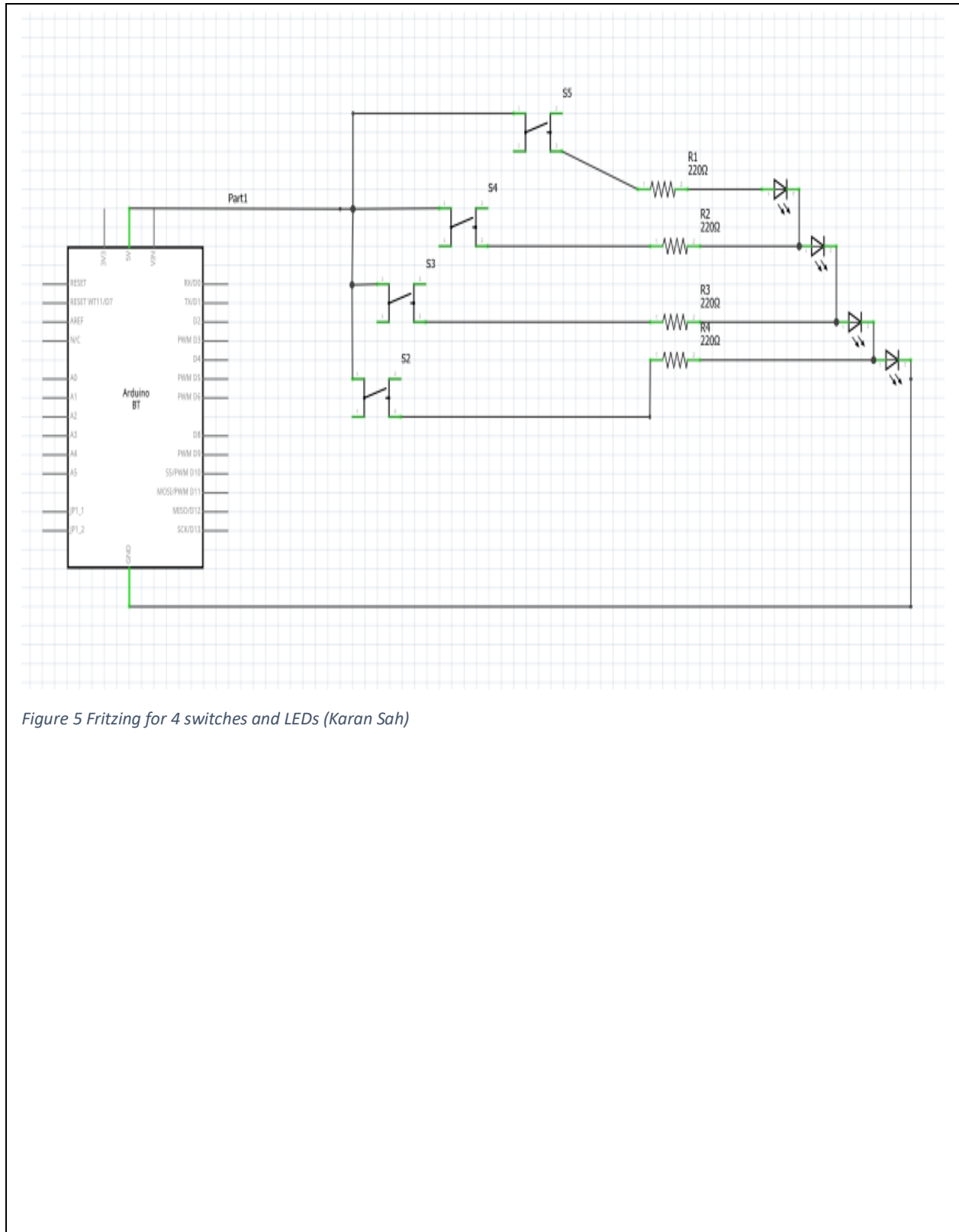
Actual measured value of current

27.3 mA

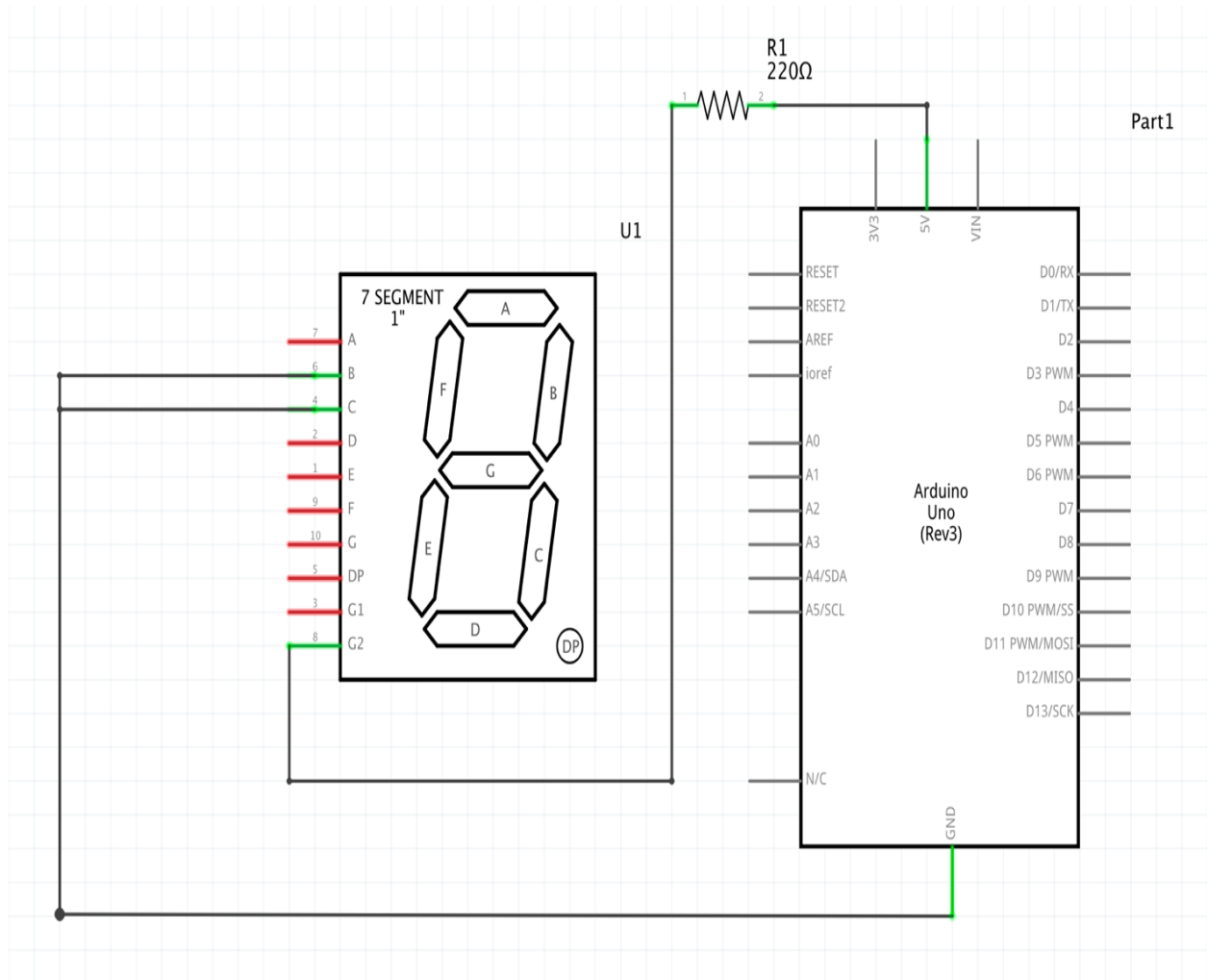
Why might they be different?

The simulation shows the ideal value to be achieved. However in real life, the value differs in comparison to that of simulation due to different factors like internal resistance of LED which oppose small currents. The length of the wire as well as its width also affects the current flowing through it. Ambient temperature is also one of the factors to affect current flow along the circuit. The material with which the wire is constructed also has an inherent resistivity.

Activity 1.8: Fritzing for 4 switches & LEDs



Activity 1.9: Fritzing for Number 0-7



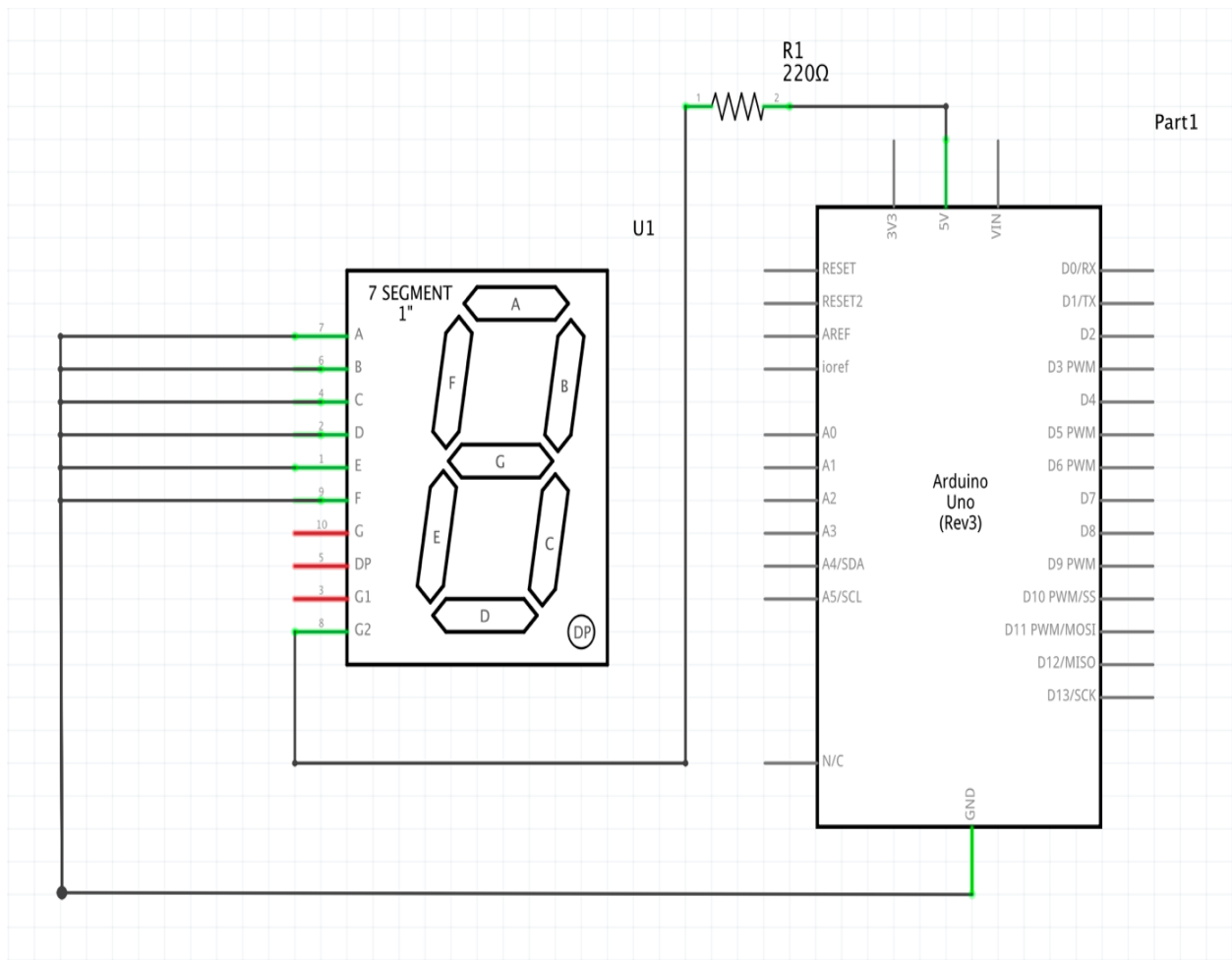


Figure 6 Fritzing for Number 0 (Karan Sah)

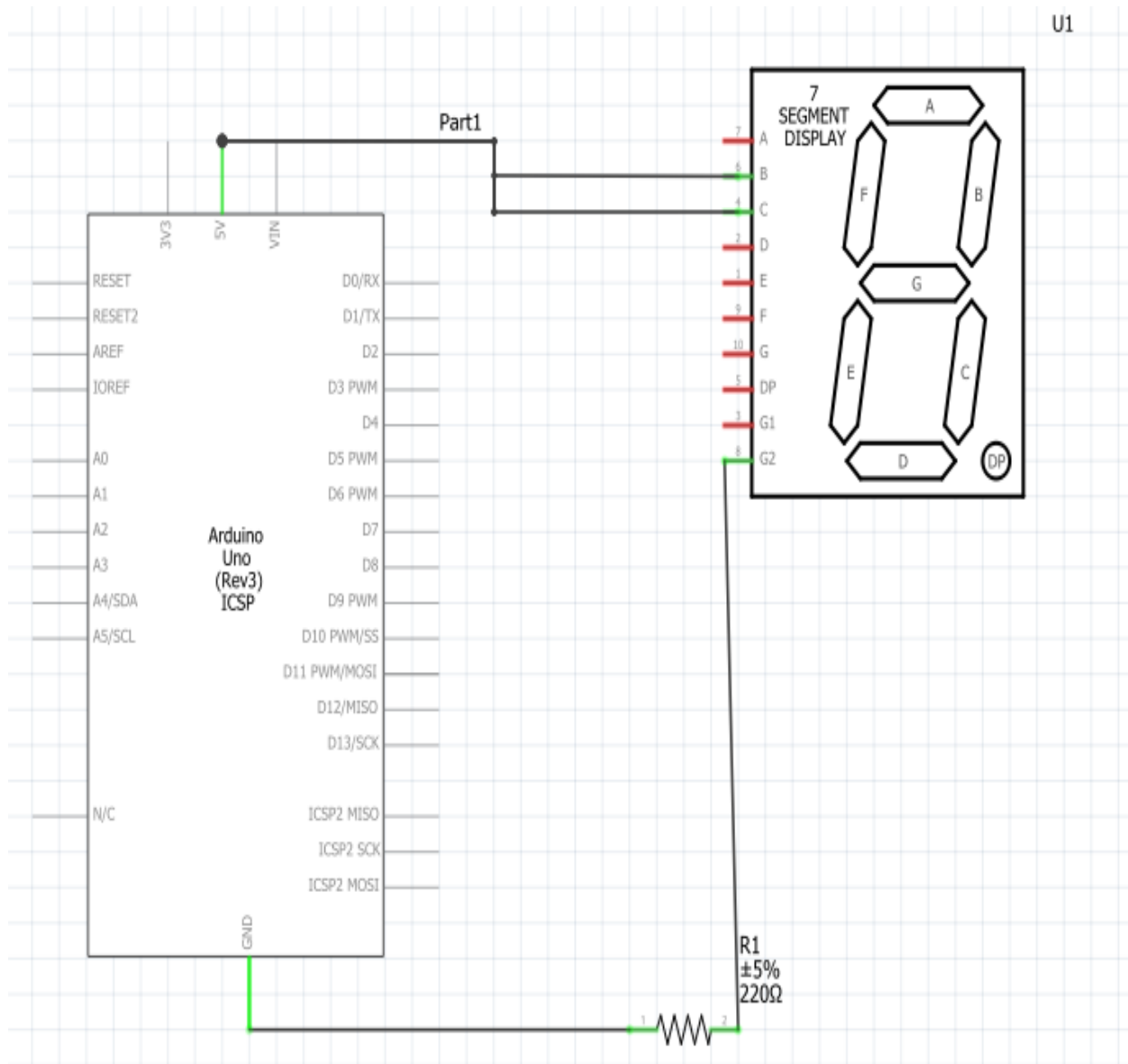


Figure 7 Fritzing for Number 1 (Karan Sah)

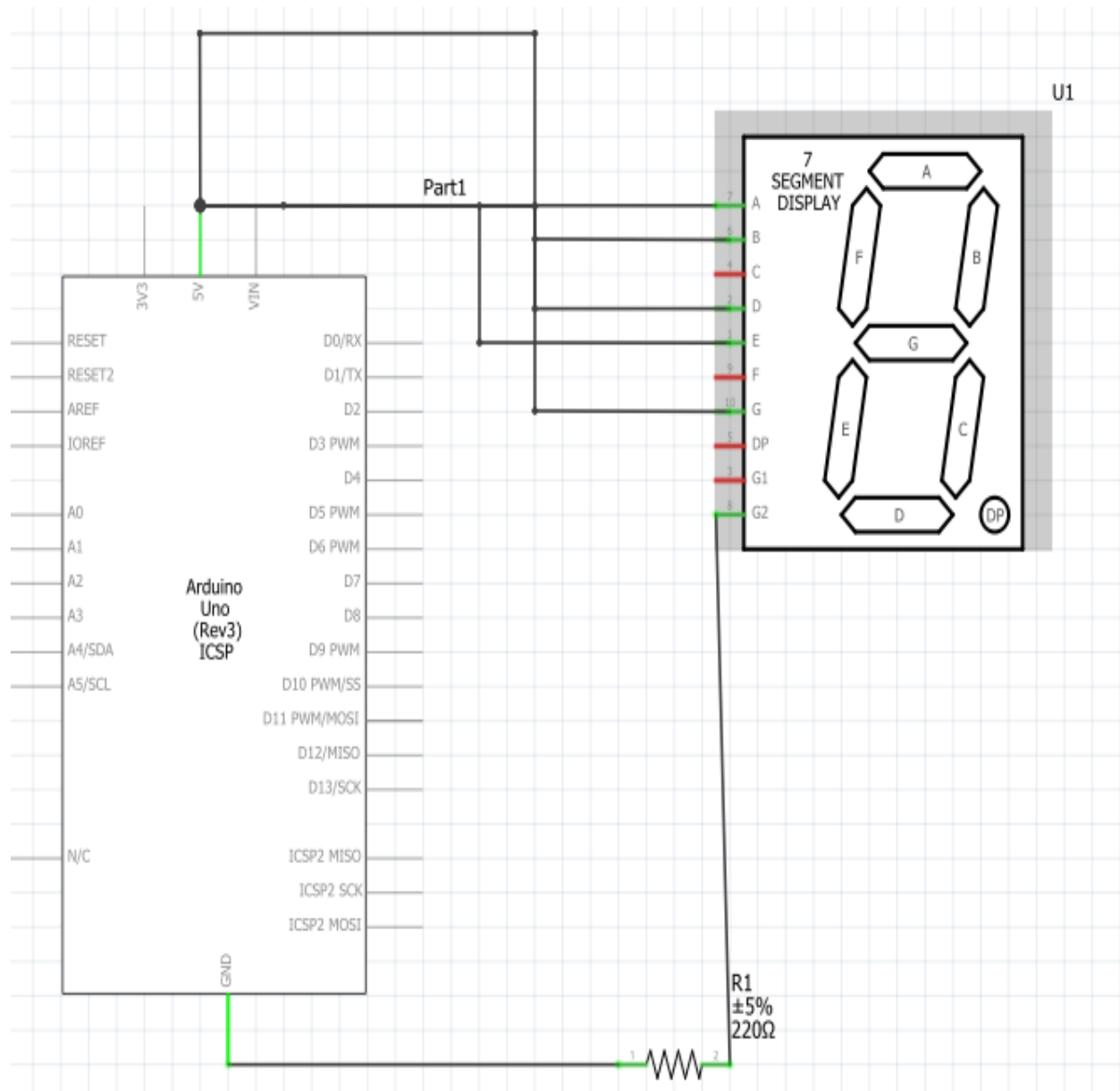


Figure 8 Fritzing for Number 2 (Karan Sah)

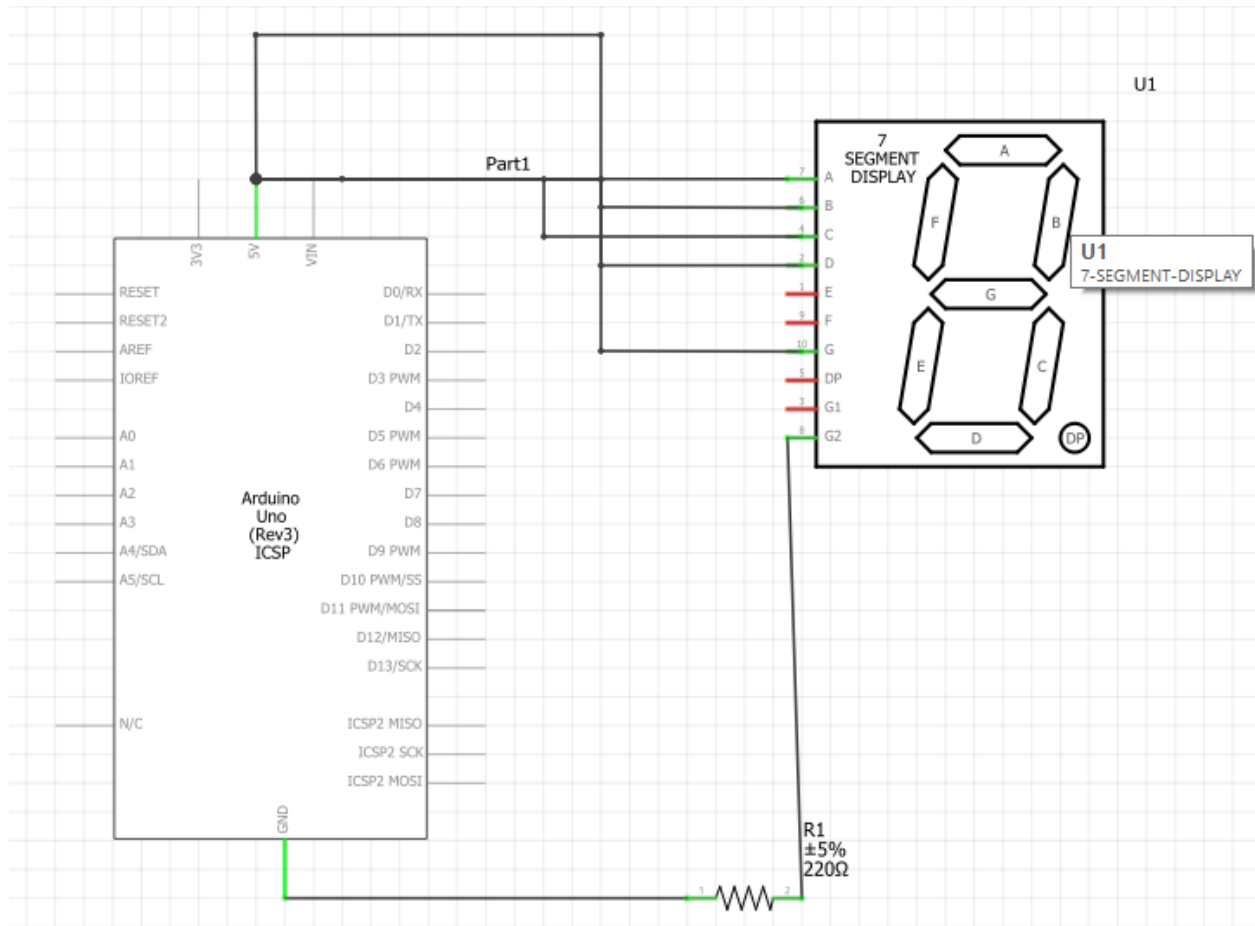


Figure 9 Fritzing for Number 3 (Karan Sah)

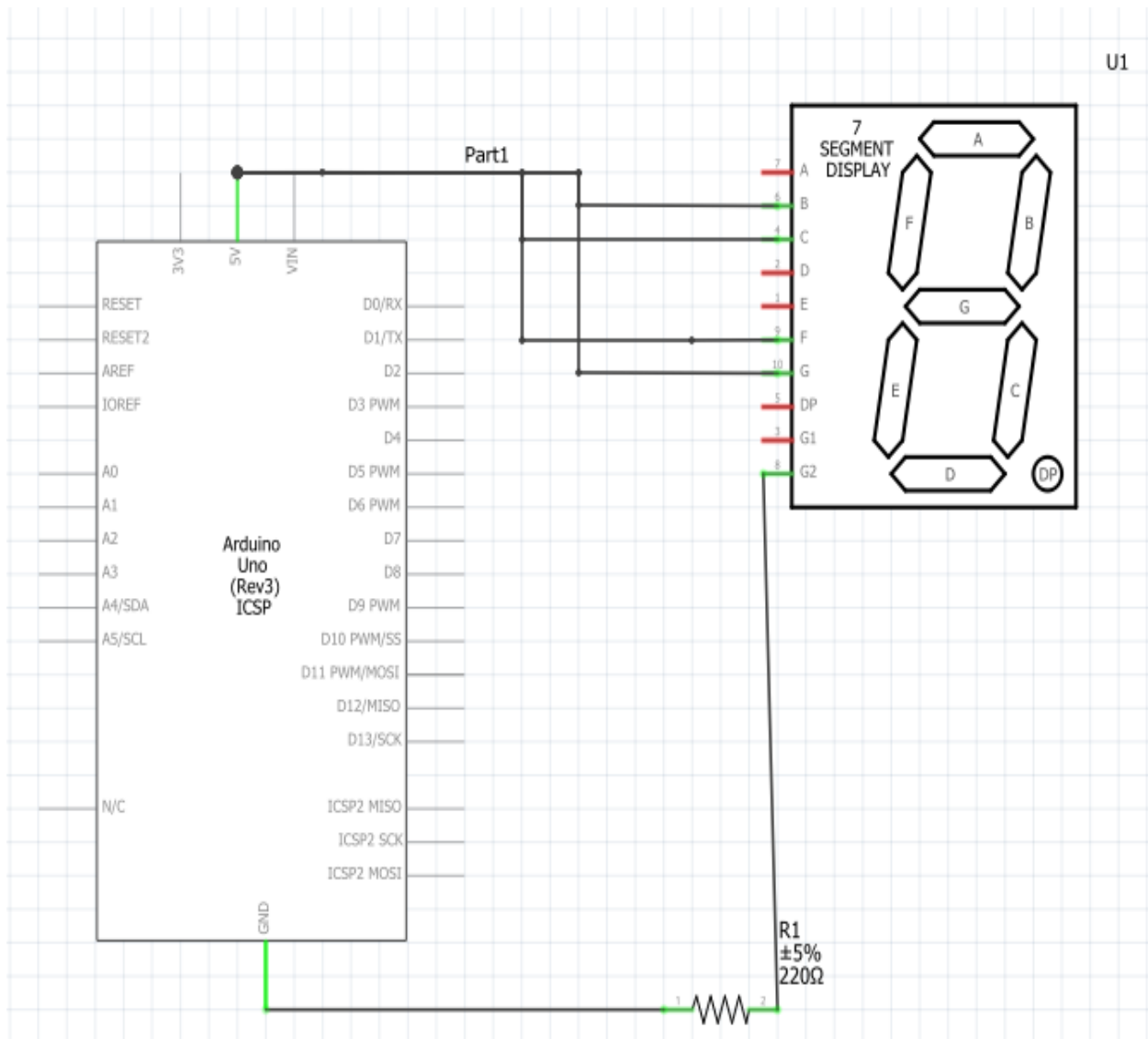


Figure 10 Fritzing for Number 4 (Karan Sah)

□

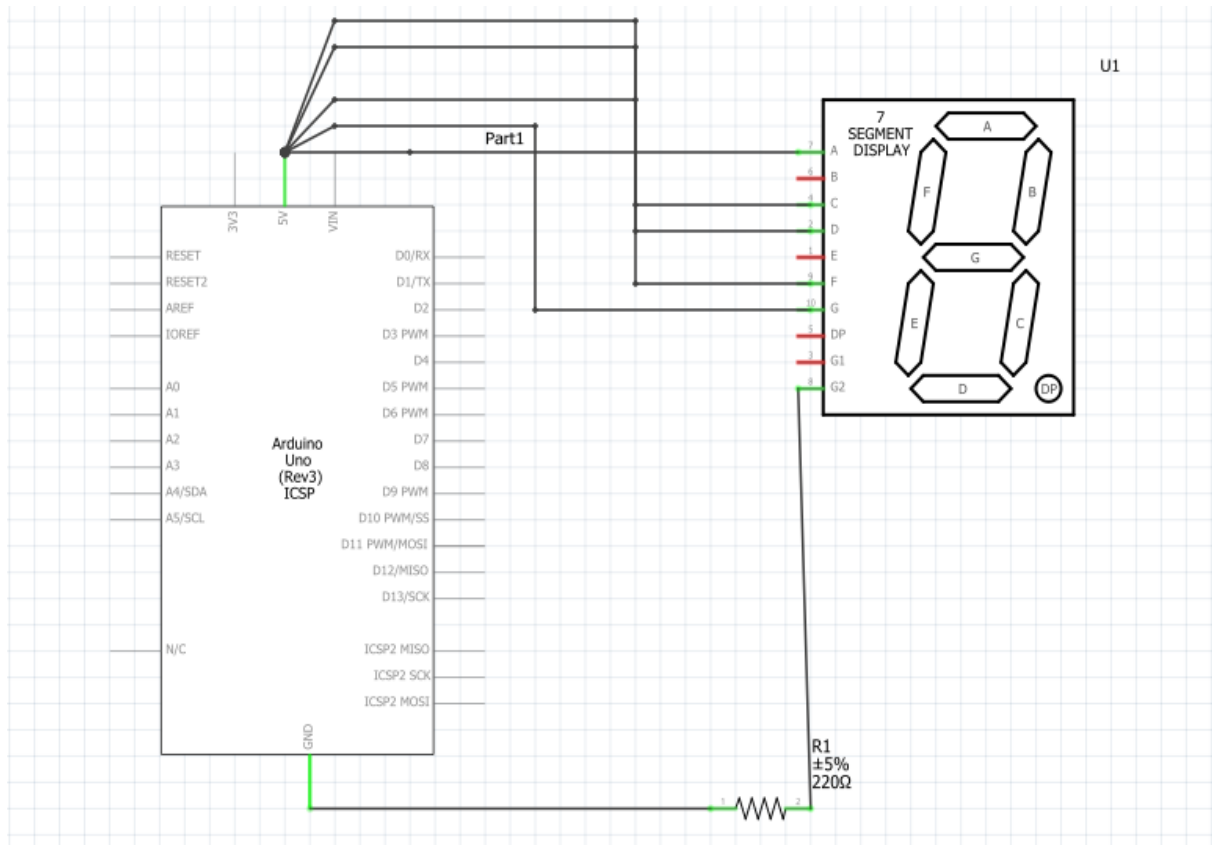


Figure 11 Fritzing for Number 5 (Karan Sah)

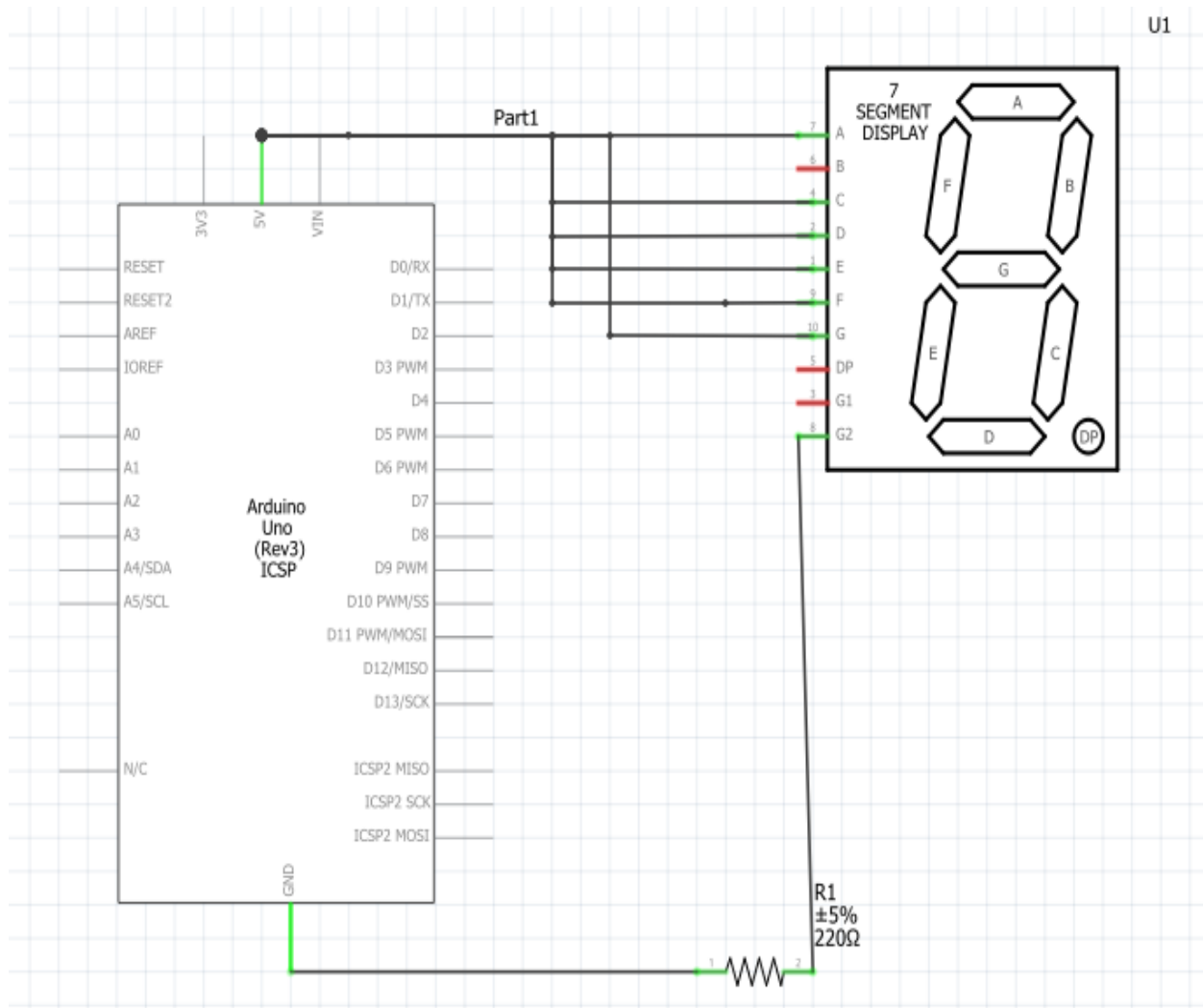


Figure 12 Fritzing for Number 6 (Karan Sah)

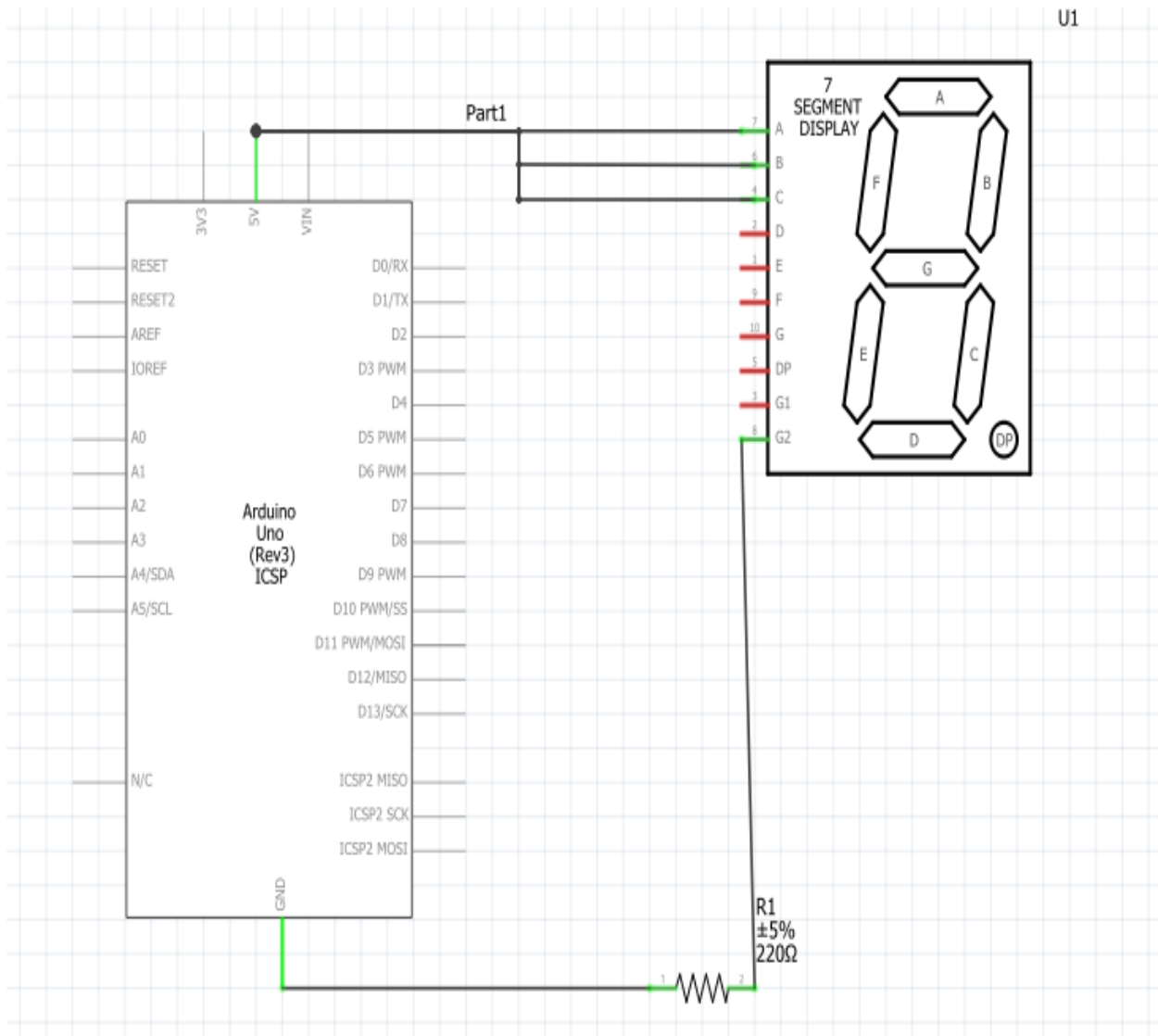


Figure 13 Fritzing for Number 7 (Karan Sah)

Workbook 2

Activity 2.1: LED Flashing to show decimal number 63 as binary.

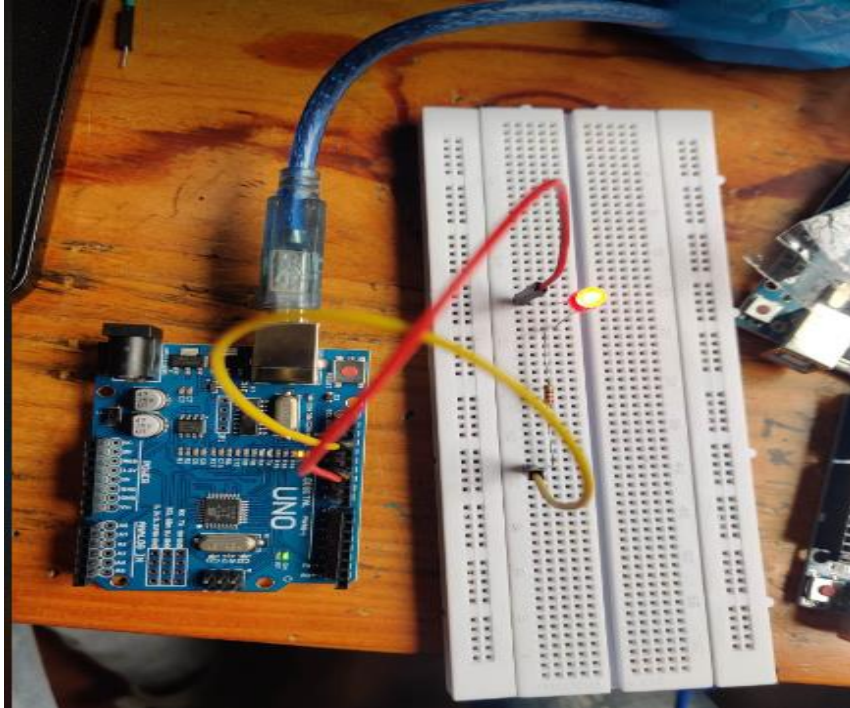


Figure 14 LED Flashing to Show Decimal Number 63 (Karan Sah)

63 as binary, including working

```
//Karan Sah
//2329860
void setup() {
  pinMode(11, OUTPUT); // setting the pin 11 as output pin by using pinMode keyword
}
// running the loop
void loop() {
  int dec=63; // Conversion of 63 to binary initialization
  while (dec>0){ //Loop till the number is 0
    int bin=dec%2; //If remainder is high or low
    digitalWrite(11, bin); //Setting the value of pin 11 to remainder
    delay(1000); // Delay of 1000 millisecond(s) or 1 second
```

```
digitalWrite(11, 0); //LED will turn off after each remainder
delay(1000); // Delay of 1000 millisecond(s) or 1 second
dec=dec/2; //Assign the new result to the loop
    }
☐ delay(5000); // Delay of 5 sec before another loop
}
```

Copy & Post your code with a suitable comment at the top of code with your name & student number 😊

Activity 2.2: 4 LED's for counting up in binary from 0 to 15.

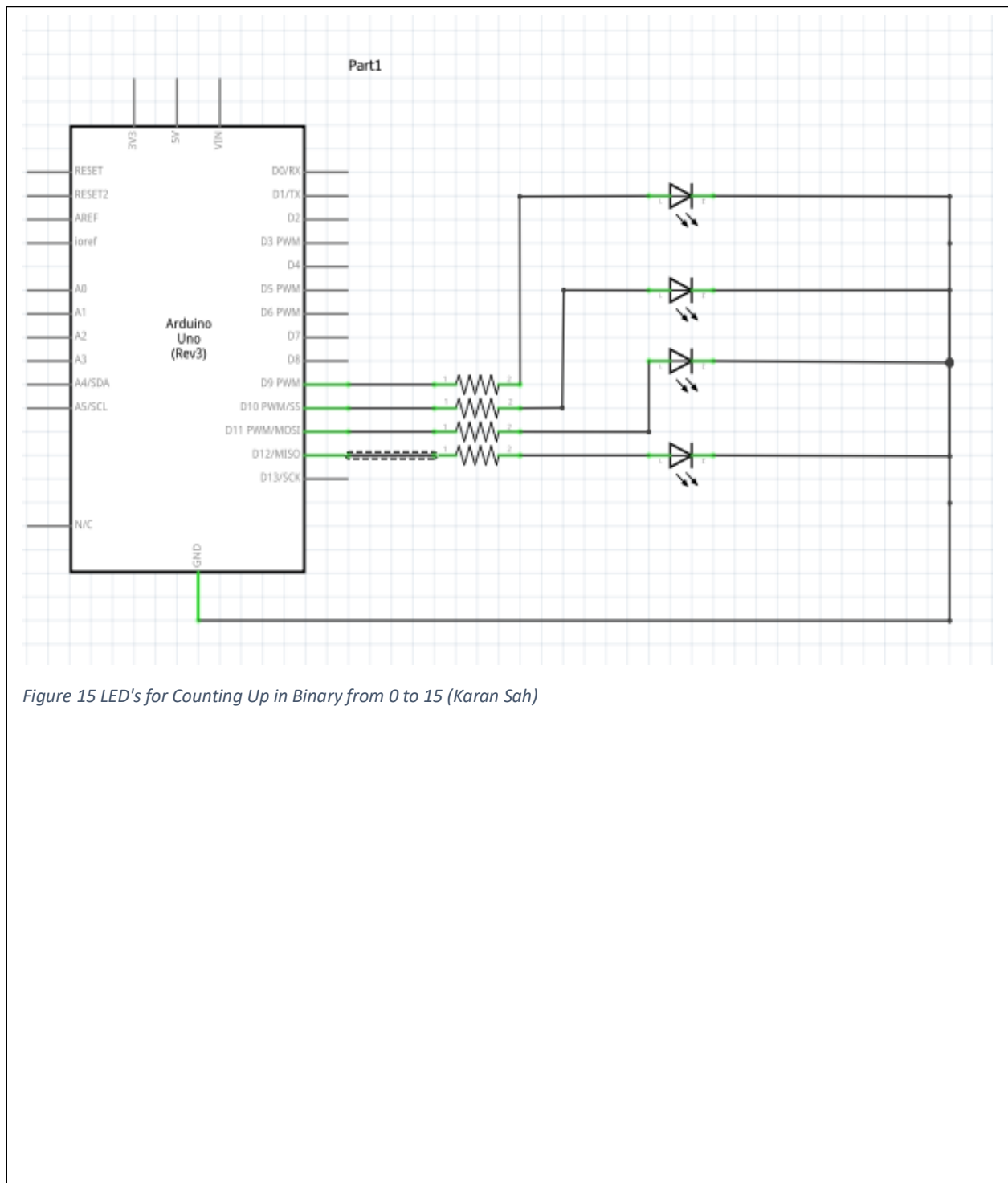


Figure 15 LED's for Counting Up in Binary from 0 to 15 (Karan Sah)

Fritzing Circuit diagram for Step 4 i.e. 4 LEDs

Arduino Program for Step 4 i.e. 4 LEDs

□

```
//Karan Sah
```

```
//2329860
```

```
//Variable declaration with one pin number assigning to one variable
```

```
int led1=9;
```

```
int led2=10;
```

```
int led3=11;
```

```
int led4=12;
```

```
void setup() {
```

```
  // Setting declared variables as output
```

```
  pinMode(led1, OUTPUT);
```

```
  pinMode(led2, OUTPUT);
```

```
  pinMode(led3, OUTPUT);
```

```
  pinMode(led4, OUTPUT);
```

```
}
```

```
void loop() {
```

```
  // To display decimal number 0 to 15 in binary
```

```
  //To call function named binary for every decimal value
```

```
  binary(0,0,0,0); //0
```

```
  binary(0,0,0,1); //1
```

```
  binary(0,0,1,0); //2
```

```
  binary(0,0,1,1); //3
```

```
  binary(0,1,0,0); //4
```

```
  binary(0,1,0,1); //5
```

```
  binary(0,1,1,0); //6
```

```

    binary(0,1,1,1); //7
    binary(1,0,0,0); //8
    binary(1,0,0,1); //9
    binary(1,0,1,0); //10
    binary(1,0,1,1); //11
    binary(1,1,0,0); //12
    binary(1,1,0,1); //13
    binary(1,1,1,0); //14
    binary(1,1,1,1); //15
}

//Function of Binary
void binary(int a, int b, int c, int d){
    //Sets the state of led of each binary value
    digitalWrite(led1,a);
    digitalWrite(led2,b);
    digitalWrite(led3,c);
    digitalWrite(led4,d);
    delay(1000); //Delay of 1000 milliseconds after each operation
}

```

Activity 2.3: Traffic Lights

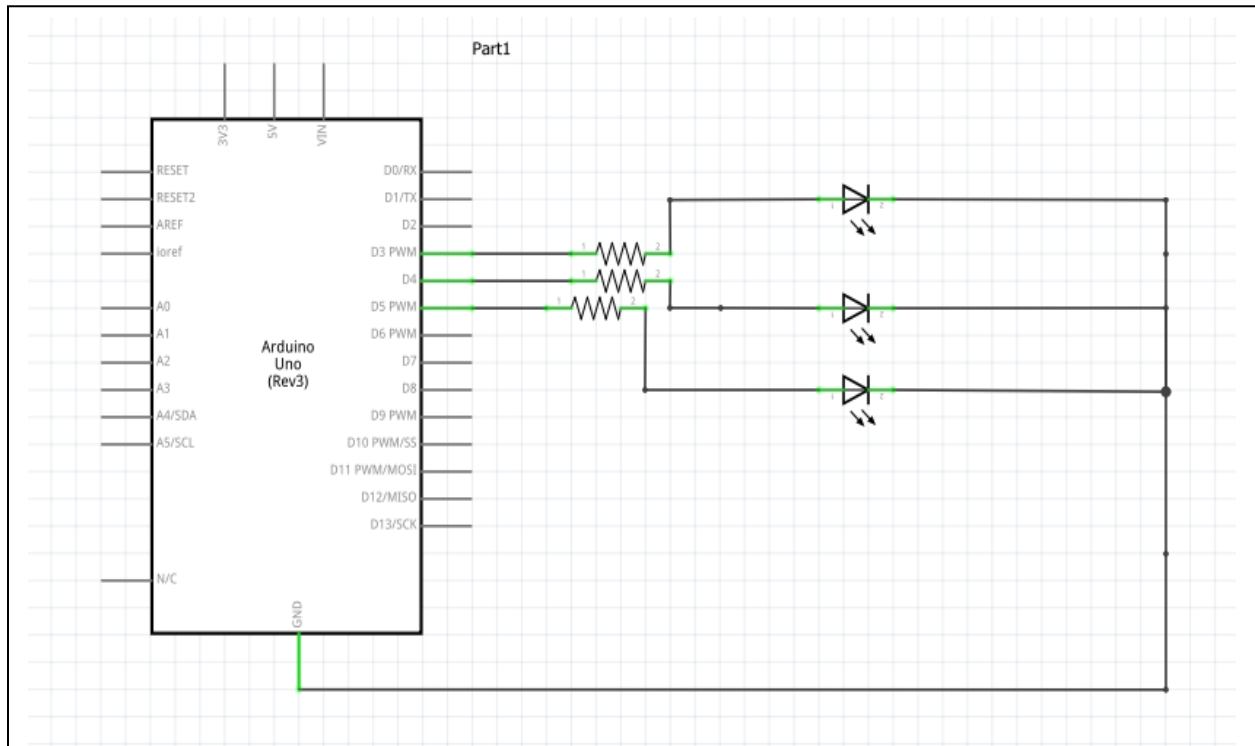


Figure 16Traffic Lights (Karan Sah)

Fritzing Circuit diagram for Step 4 i.e. 4 LEDs

Arduino Program for Step 4 i.e. 4 LEDs

□

```
//Karan Sah
```

```
//2329860
```

```
int led1=3;//Red light
```

```
int led2=4;//Yellow light
```

```
int led3=5;//Green light
```

```
//setting pins for the traffic lights to output
```

```
void setup()
```

```
{
```

```
  pinMode(led1, OUTPUT);
```

```
  pinMode(led2, OUTPUT);
```

```
  pinMode(led3, OUTPUT);
```

```
}
```

```
void loop()
```

```
{
```

```
  int a=0;//counter to iterate for 5 times
```

```
  do{
```

```
    digitalWrite(led1,HIGH); // lights Red LED
```

```
    delay(3000); // Delay of 3000 millisecond(s) or 3 seconds
```

```
    digitalWrite(led1,HIGH); digitalWrite(led2,HIGH); //Lights Red and Yellow LED
```

```
    delay(2000); // Delay of 2000 millisecond(s) or two seconds
```

```
    digitalWrite(led1,LOW); digitalWrite(led2,LOW); digitalWrite(led3,HIGH); //Lights Green LED
```

```
    delay(3000); // Wait for 3000 millisecond(s) or 3 seconds
```

```
    a=a+1;
```

```
  }while(a<5);
```

```
  delay(5000);//Delay of 5000 millisecond(s) or 5 seconds between loops
```

```
}  
  
//Function to light LEDs along with instructions.  
  
void main(int a,int b, int c){  
    digitalWrite(led1, a);  
    digitalWrite(led2, b);  
    digitalWrite(led3, c);  
}
```

Workbook 3

Activity 3.1: Circuit Diagram of Button & LED

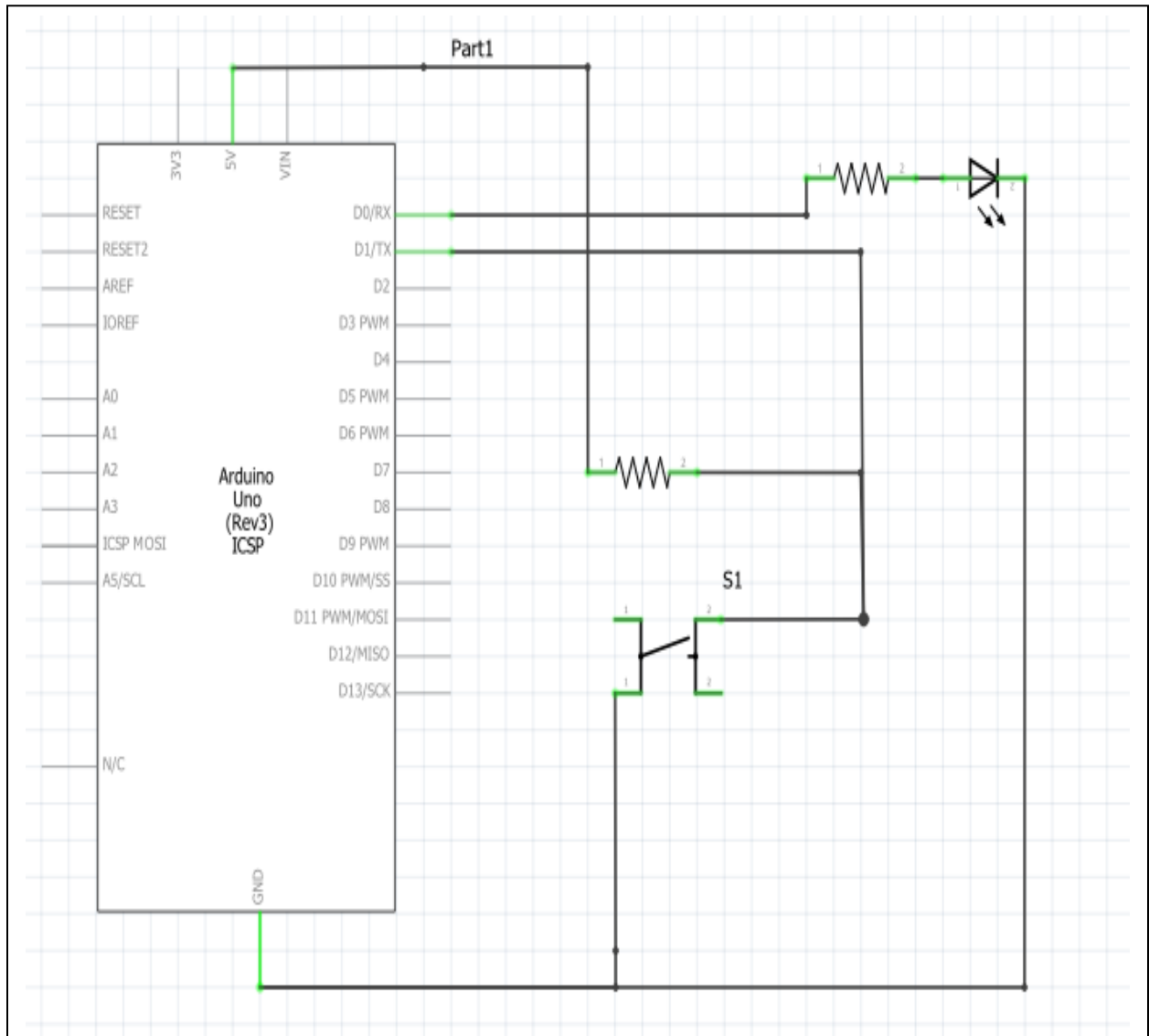


Figure 17 Pull up Resistor (karan Sah)

Fritzing

Activity 3.2: 3 Switches & Led

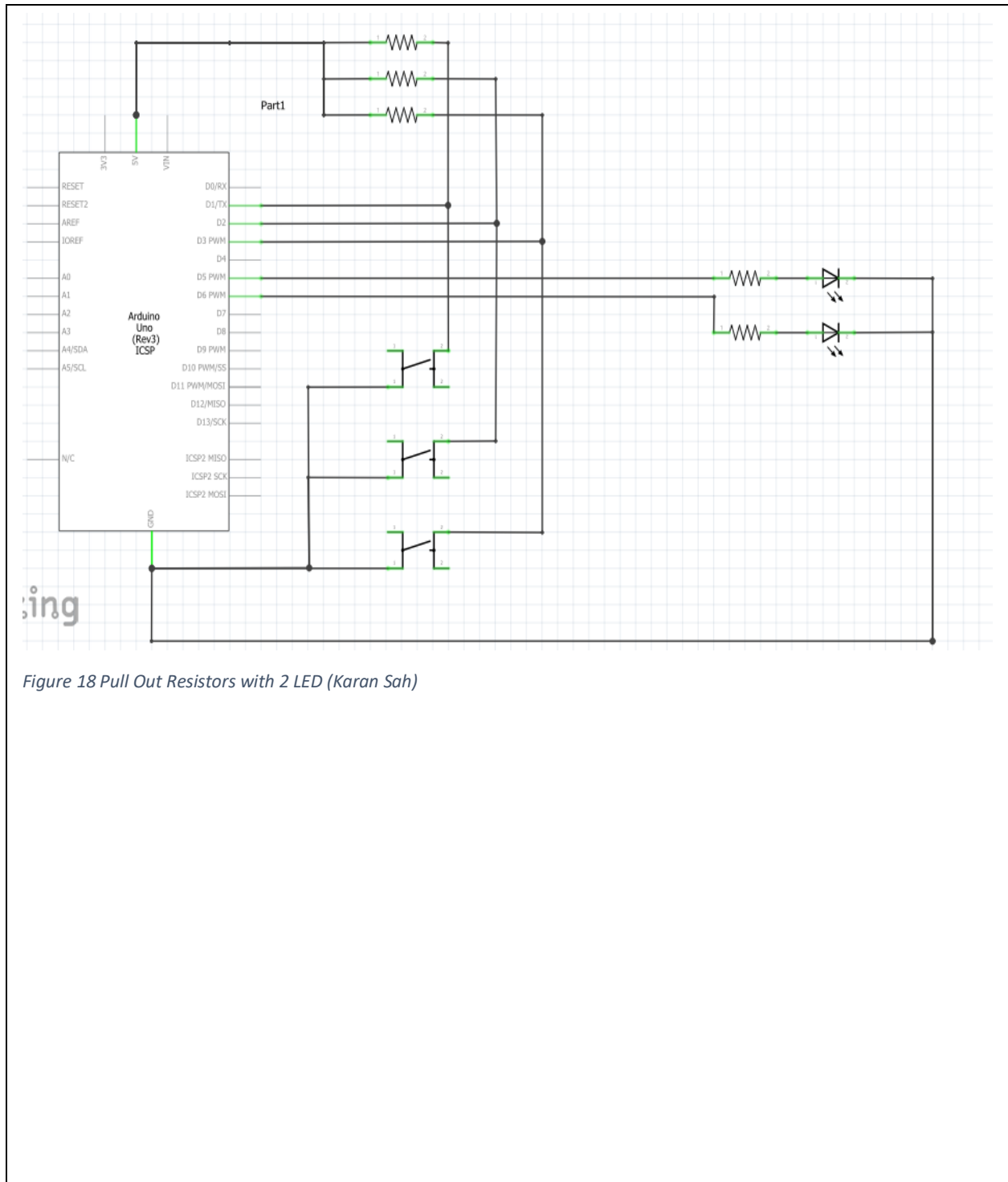


Figure 18 Pull Out Resistors with 2 LED (Karan Sah)

Fritzing Circuit Diagram

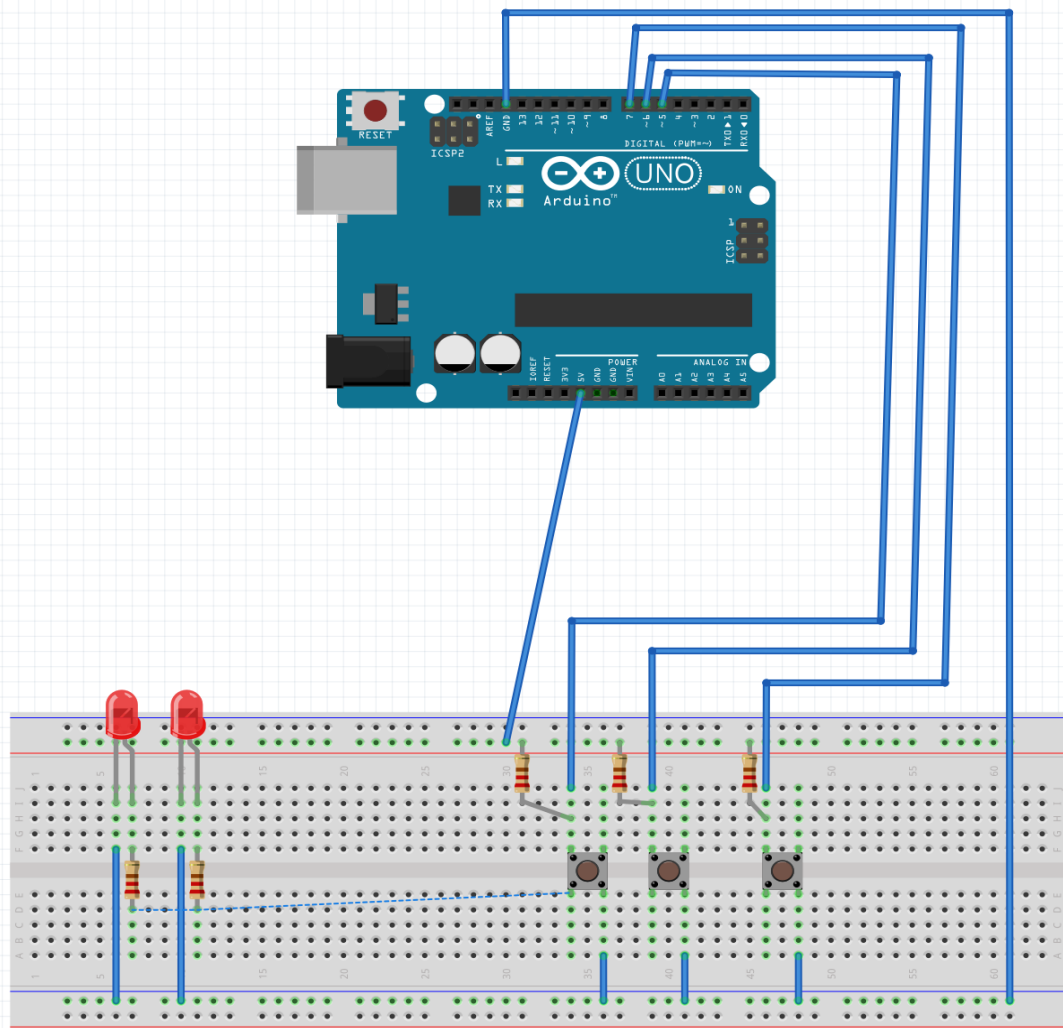


Figure 19 3 Switches and 2 LEDs Arduino Setup (Karan Sah)

Arduino Program

□

//Karan Sah

//2329860

//C++

void setup()

{

// setting pin 5 and 6 to OUTPUT mode

pinMode(5, OUTPUT);

```

pinMode(6, OUTPUT);

//setting pin 1,2 and 3 to INPUT_PULLUP mode
pinMode(1, INPUT_PULLUP);
pinMode(2, INPUT_PULLUP);
pinMode(3, INPUT_PULLUP);
}

void loop()
{
    //Storing the states of the switches to the variables
    int switchState=digitalRead(1);// reads the state of pin 1 and storing it in switchState
    int switchState1=digitalRead(2);// reads the state of pin 2 and storing it in switchState1
    int switchState2=digitalRead(3);// reads the state of pin 3 and storing it in switchState2
    //checks if 1st switch is pressed or not based on the state of switchState
    if (switchState== 0){
        //lights led at pin 5 for 1 sec
        digitalWrite(5,1);
        delay(1000);
    }else{
        digitalWrite(5,0);//sets led off(pin 5 to low)
    }

    //checks if 2nd switch is pressed or not based on the state of switchState1
    if (switchState1== 0){
        //lights led at pin 6 for 2 sec
        digitalWrite(6,1);
        delay(2000);
    }else{
        digitalWrite(6,0);//sets led off(pin 6 to low)
    }
}

```

```
//checks if 3rd switch is pressed or not based on the state of switchState2
if (switchState2== 0){
    //lights led at pin 5 for 3 sec
    digitalWrite(5,1);
    delay(3000);
}else{
    digitalWrite(5,0);//sets led off(pin 5 to low)
}
}
```

Activity 3.3: 8 Buttons & LEDs (SWITCH STATEMENTS)

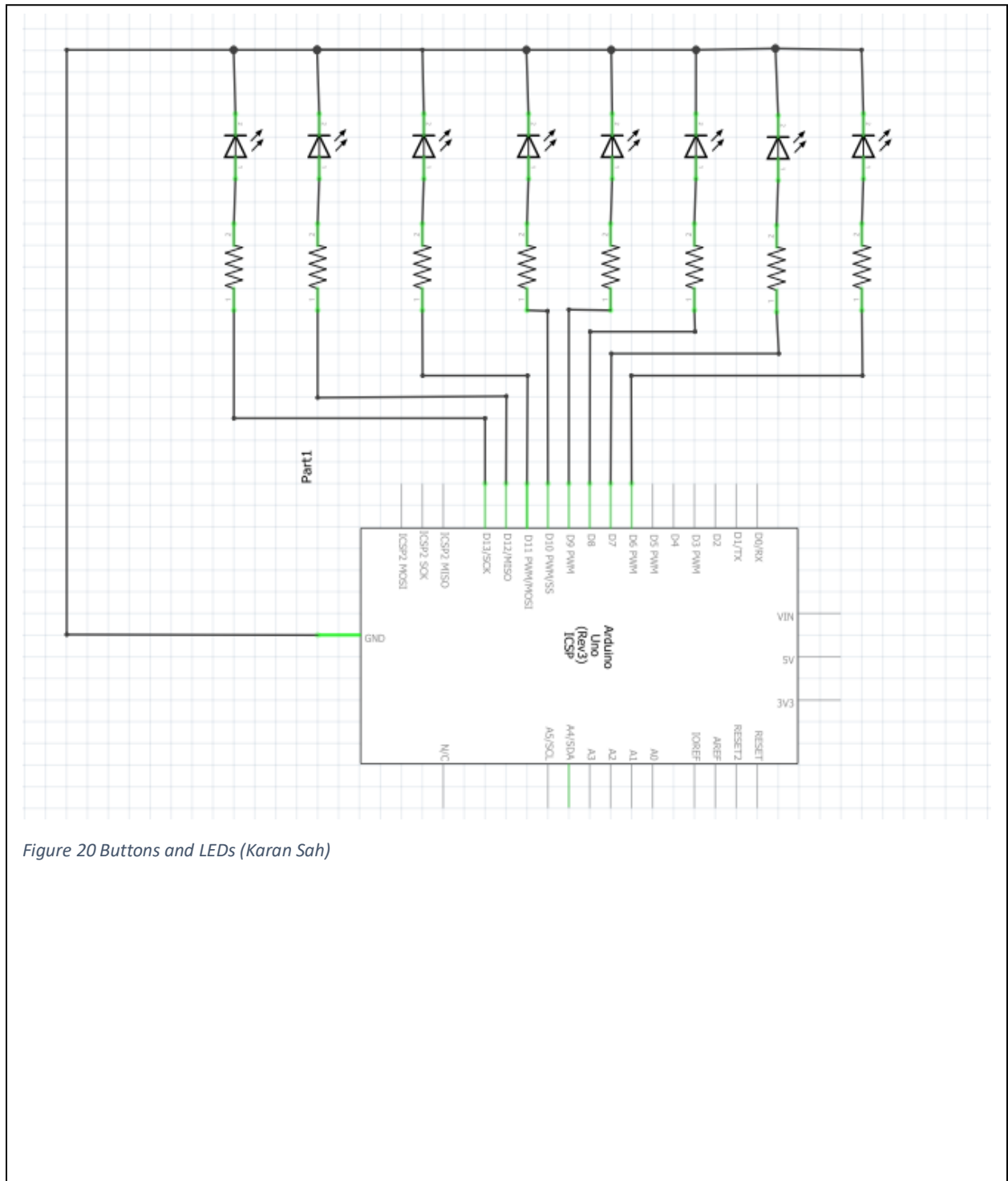


Figure 20 Buttons and LEDs (Karan Sah)

Fritzing

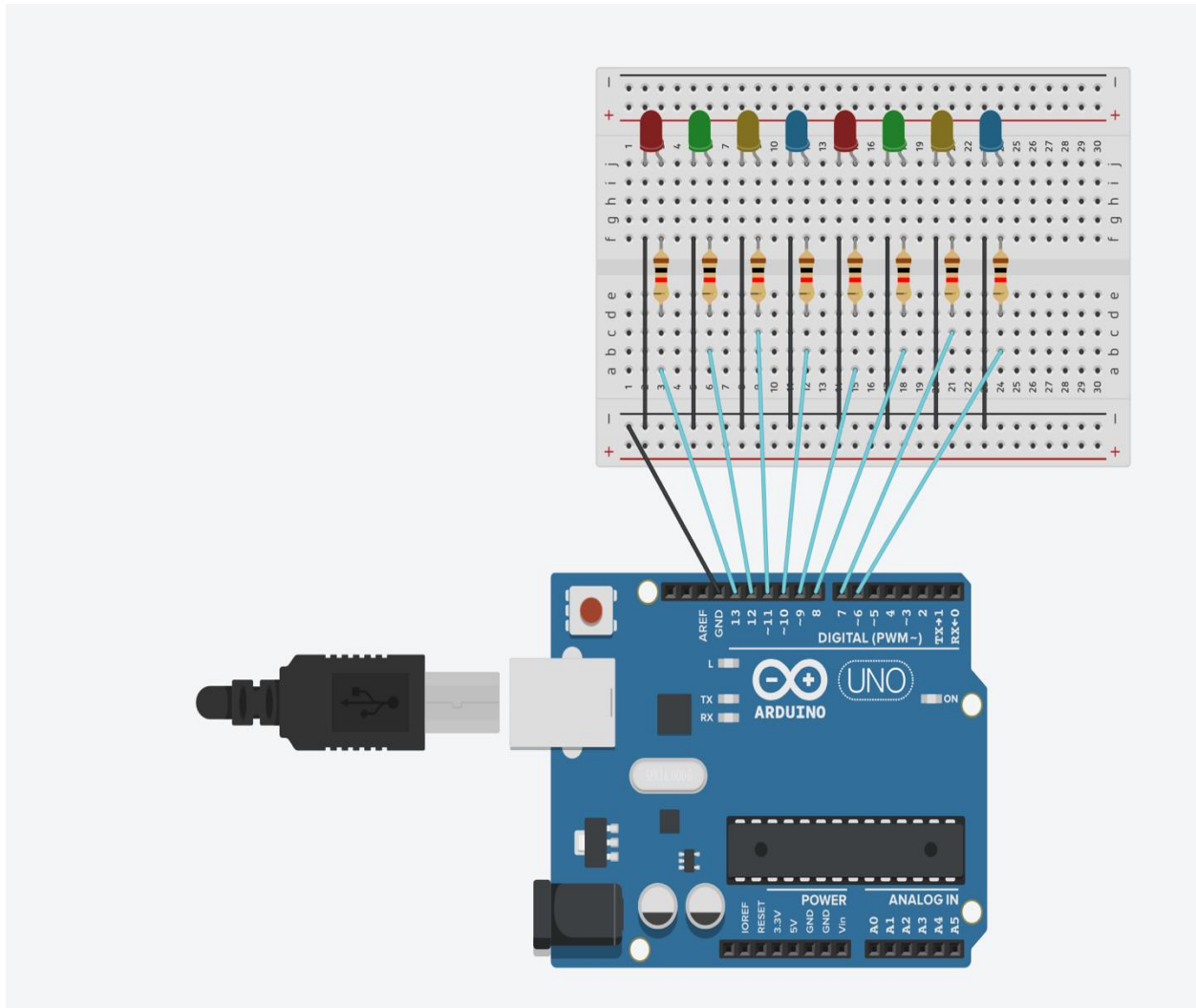


Figure 21 8 LEDs Representing 8-bit Binary Number: Arduino Setup (Karan Sah)

Arduino Program

```
//Karan Sah
//2329860
//Array to store pin numbers of the 8 LEDs starting from 6 to 13.
int led[8] = {6, 7, 8, 9, 10, 11, 12, 13};
//Values converting to Binary
int value = 255;

void setup()
{
```

```

// for loop to set the pin modes of the LEDs to OUTPUT
int i;
for (i = 0; i < 8 ; i++)//running the loop till the i is less than 8
    pinMode(led[i], OUTPUT);
}

void loop()
{
    // calls the displayOutput function with the value to be displayed
    displayOutput(value);
    delay(1000); // Delay for 1 second
}

void displayOutput(int val)
// function to display the binary representation of a decimal number on the LEDs
{
    if (val<=255){//checks if the value of val is smaller than or equal to 255 or not
        int i;
        for (i = 0; i < 8; i++)
        {
            if (val % 2 == 0)// checks if the remainder of val/2 is 0
            {
                digitalWrite(led[i], LOW);// sets the LED pin from "i" to LOW (OFF)
            }
            else
            {
                digitalWrite(led[i], HIGH);// sets the LED pin from "i" to HIGH (ON)
            }
            val = val / 2; // divides val by 2 to get the next binary digit
        }
    }
}

```

```
    }  
}  
else//if val is greater than 255 all LEDs pin sets LOW  
{  
    int i;  
    for (i = 0; i < 8; i++){  
        digitalWrite(led[i], LOW);  
    }  
}  
}
```

Workbook 4

Activity 4.1: Serial Port

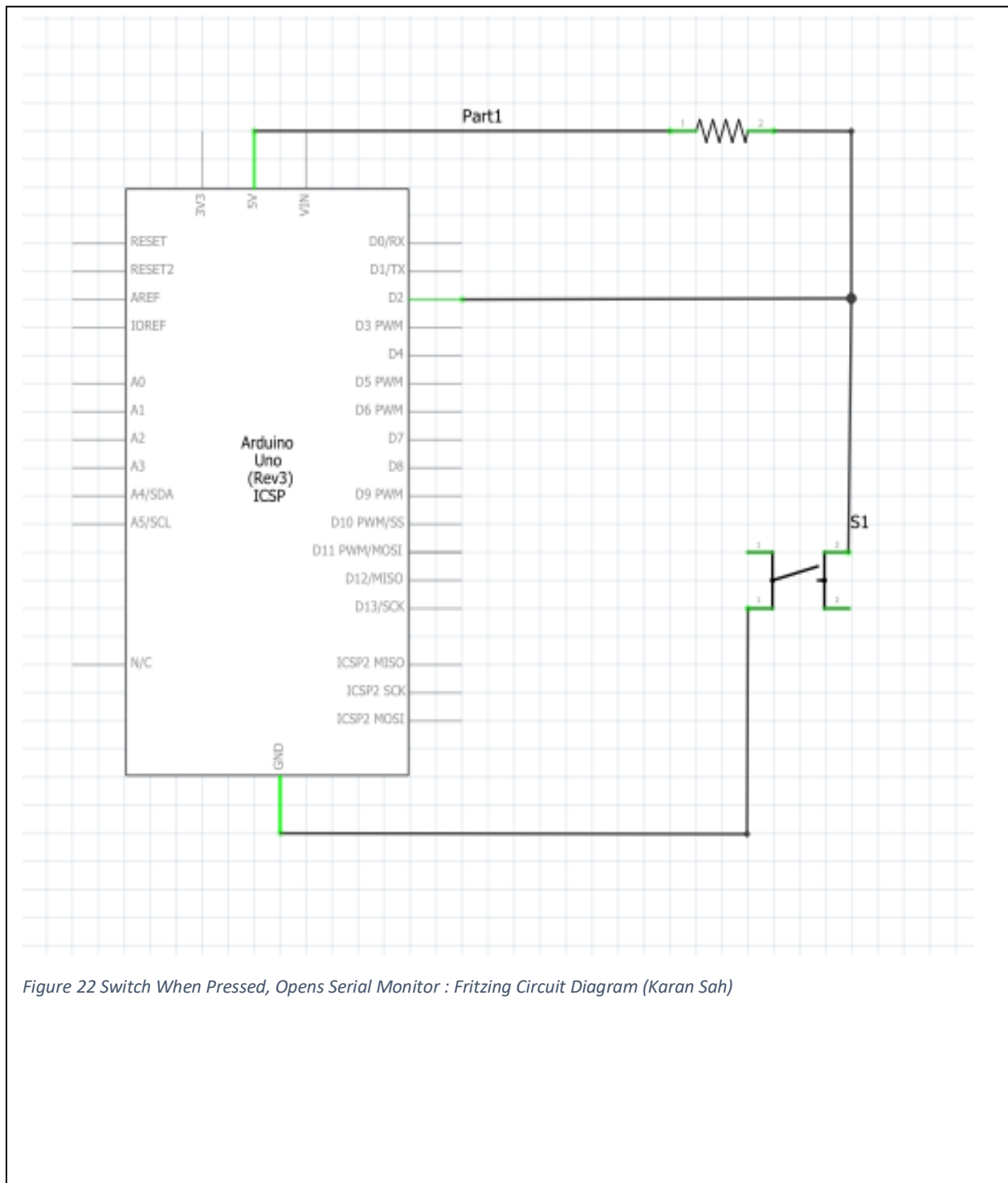


Figure 22 Switch When Pressed, Opens Serial Monitor : Fritzing Circuit Diagram (Karan Sah)

Fritzing

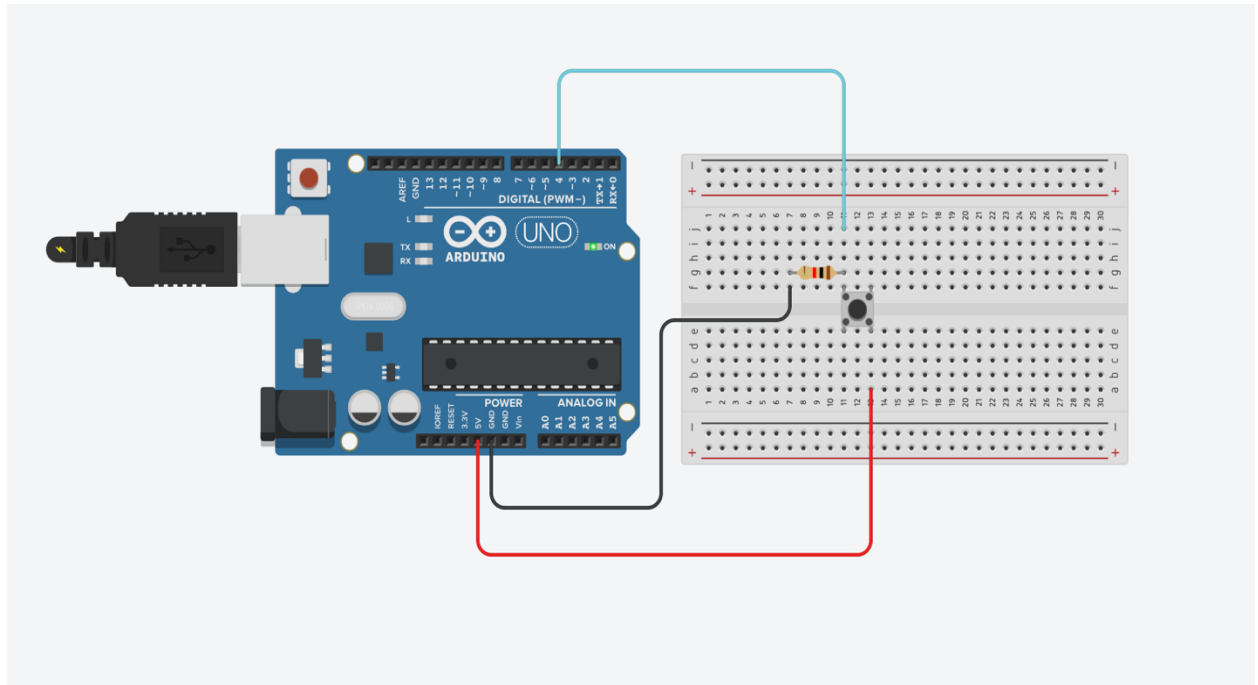


Figure 23 Switch to Open Serial Monitor : Arduino Setup (Karan Sah)

```
//Karan Sah
//2329860
long id=2329860;//initializing id
void setup() {
  // Initialize the serial communication with a baud rate of 9600
  Serial.begin(9600);
}
void loop() {
  //store the state of pin 2 connected to the pull up resistor
  int switchState=digitalRead(2);
  //if switch is pressed
  if (switchState==0){
    // Print and displays the student name and ID to the Serial Monitor
    Serial.print("Student Name:");
    Serial.println("Karan Sah");
    Serial.print("Student ID:");
    Serial.println(id);
    //delay for 500 milliseconds
    delay(500);
  }
```

Arduino Program



Serial Monitor

- Student Name:Karan Sah
- Student ID:2329860
- Student Name:Karan Sah

Figure 24 Output in Serial Monitor (Karan Sah)

Screen Shot of Serial Port

-

Activity 4.2: Serial Port binary to decimal

Code

```
//Karan Sah
//2329860
// declaring an array of 8 input pins
int Inputs[8] = {6, 7, 8, 9, 10, 11, 12, 13};
//declaring a variable to be used in both setup and loop function
int i;
void setup()
{
    //sets the input pins in the array as inputs
    for (i = 0; i < 8; i++)
    {
        pinMode(Inputs[i], INPUT);
    }
    // Start the serial communication at 9600 baud rate
    Serial.begin(9600);
}
void loop()
{
```



```

int n;//to store the switch input
float result = 0;//to store decimal result

    //checks all the input pins value (either 1 or 0) representing binary digits
for (i = 0; i < 8; i++)//iterate through the array
{
    int switchState = digitalRead(Inputs[i]); //stores the state of input pin
    //if condition to get the binary value
    if (switchState == HIGH)
    {
        n = 1;//if input pin is high n is 1
    }
    else
    {
        n = 0;//if input pin is low n is 2
    }
    result = result + n * pow(2, i);
}

//prints the result in the serial monitor
Serial.println("The decimal representation is : ");
Serial.println(result);
delay(1000);//delay of 1 second after each loop
}

```

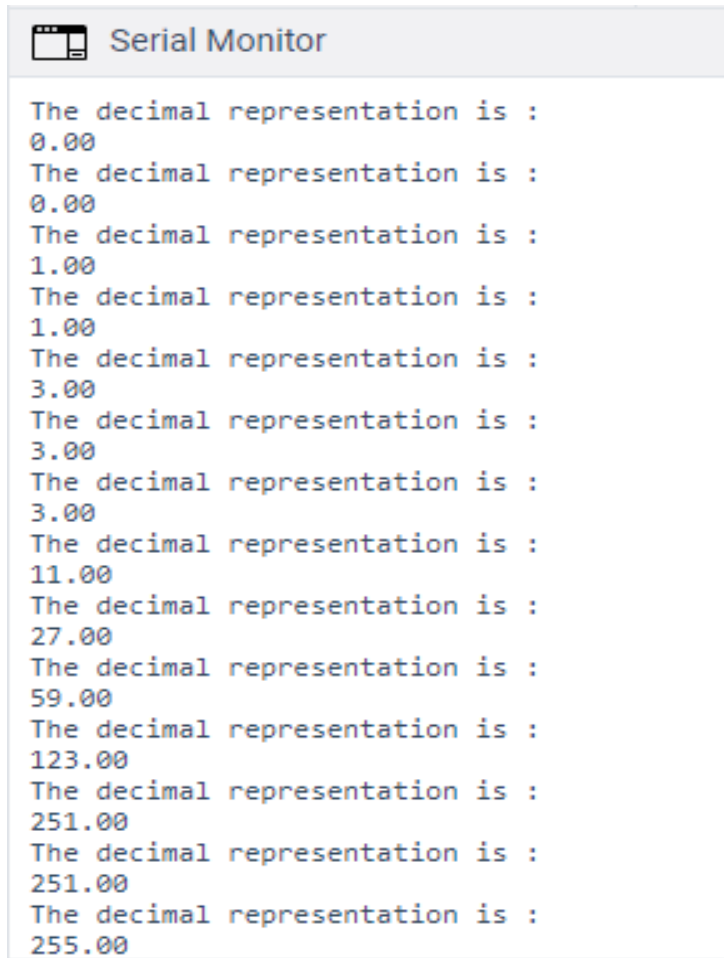


Figure 25Serial Port Output Binary to Decimal Using 8 Switch (Karan Sah)

Screen Shot of Serial Port

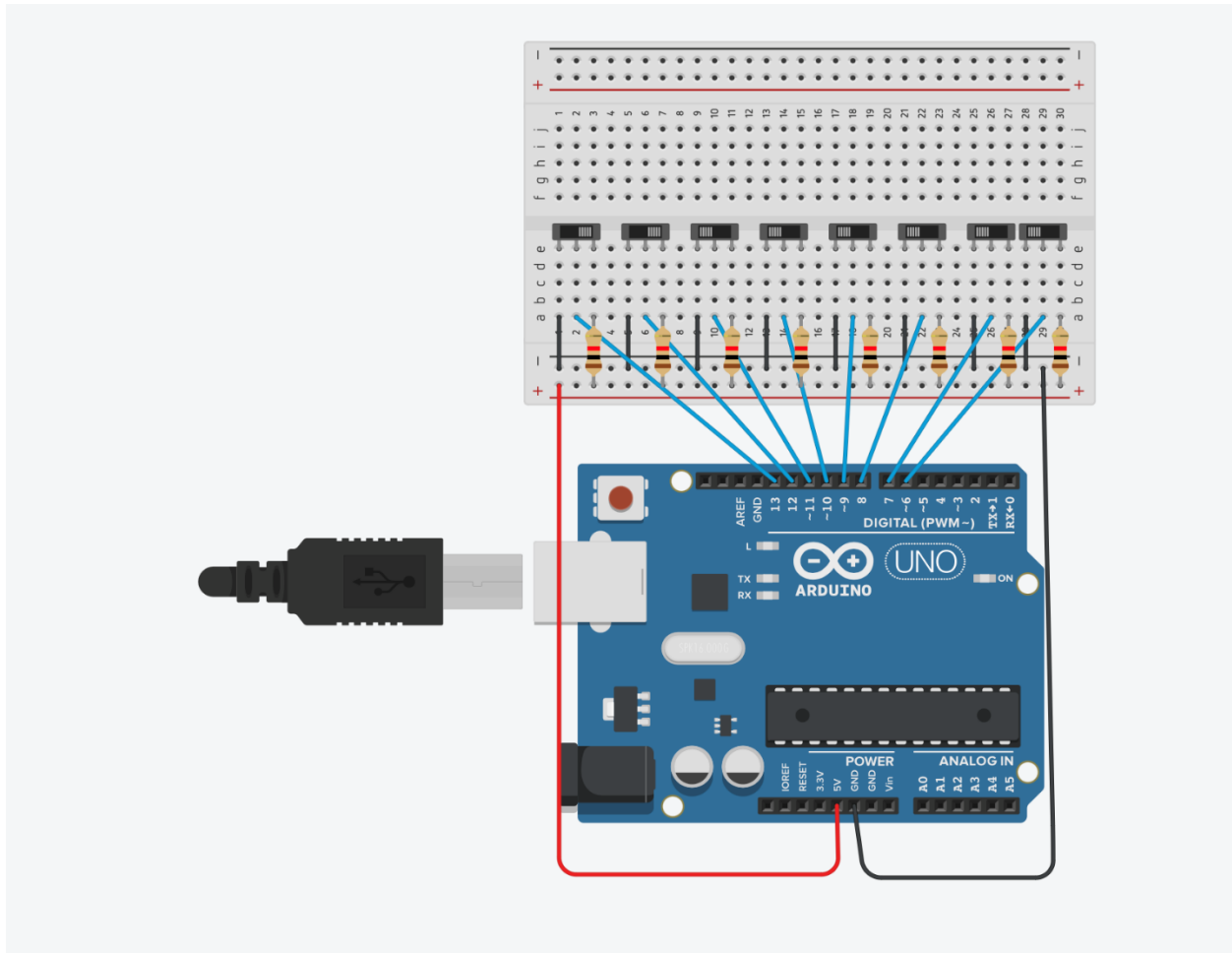


Figure 26 Decimal Converter Using 8 Switches : Arduino Circuit SetUp (karan Sah)

Activity 4.3: Calibrating Analogue Information

//Karan Sah

//2329860

int switchInput = 12;

int pot;

void setup()

{

Serial.begin(9600);

```

pinMode(A0, INPUT);
pinMode(switchInput, INPUT);
}

void loop()
{
  int sensor = digitalRead(switchInput);
  if (sensor == HIGH)
  {
    float Vin = 5.0;
    float maxOutput = 1023;

    pot = analogRead(A0); //Reading the value of A0 and store the value in variable pot = map(pot, 0,
    1023, 0, 255) directly

    float voltage = pot * (Vin/maxOutput);
    Serial.print("Value of Potentiometer: ");
    Serial.println(pot); //Sends the value of pot to Serial monitor
    Serial.print("Voltage: ");
    Serial.println(voltage); //Printing the value of voltage on Serial monitor
    float resistance = 10000 - (voltage * 10000/Vin); //converting into ohm
    Serial.print("Resistance: ");
    Serial.println(resistance); //Printing resistance on Serial monitor
    delay(1000); //delay of 1 second
  }
}

```

Code

Pot Resistance Clockwise

10 kohm

Pot Resistance Anti-clockwise

0 kohm

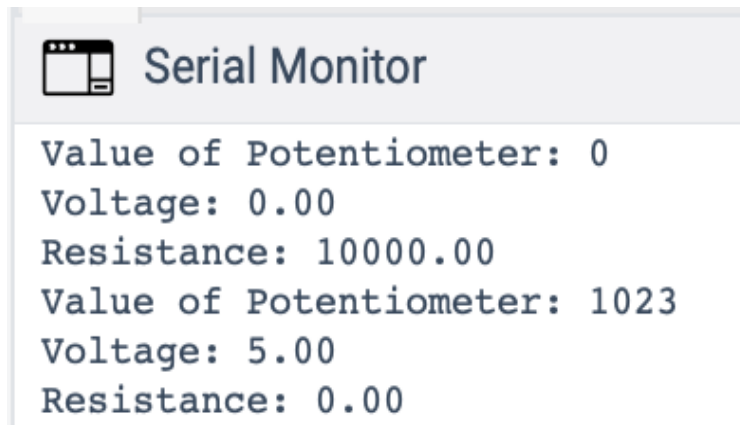


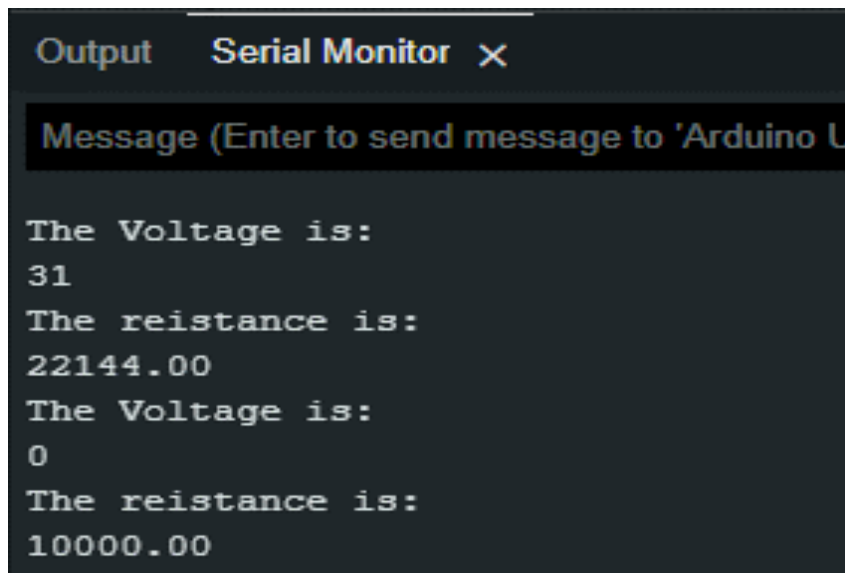
Figure 27 Potentiometer Resistance Clockwise and Anti- Clockwise Displayed in Serial Monitor (Karan Sah)

Sample of Values

Pot Resistance against Voltage change

Pot Resistance	Voltage Measured
0.4 K Ω	4.8 V
8.79 K Ω	0.6V
5.39 K Ω	2.3 V
3.59 K Ω	3.2 V
7.2 K Ω	1.4 V

Screen Shot of Meaningful Serial Port Output, not just numbers



The screenshot shows a 'Serial Monitor' window with a title bar that includes 'Output', 'Serial Monitor', and a close button. Below the title bar is a text input field with the placeholder text 'Message (Enter to send message to 'Arduino U...'. The main area of the window displays the following text in a monospaced font:

```
The Voltage is:
31
The reistance is:
22144.00
The Voltage is:
0
The reistance is:
10000.00
```

Figure 28 Serial Port Output of Potentiometer's Resistance Against Voltage Change (Karan Sah)

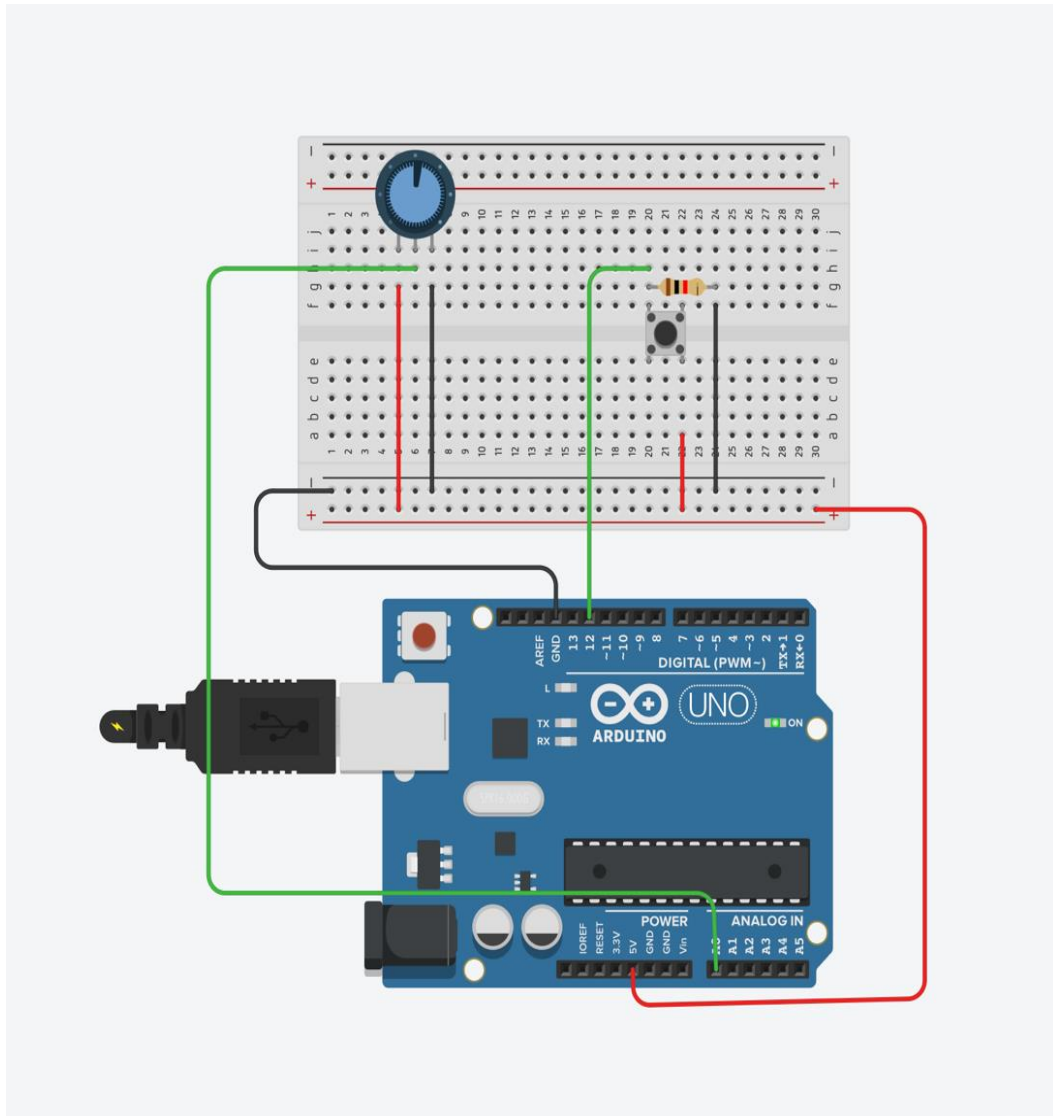


Figure 29 Serial Output of Potentiometer's resistance Against Voltage Change : Arduino Circuit Setup (Karan Sah)

Activity 4.4: Temperature Sensor & Serial Port

//Karan Sah

//2329860

int tempPin = A0; // Analog input pin for the temperature sensor

void setup() {

Serial.begin(9600); // Initialize serial communication at 9600 bits per second

```

pinMode(2, INPUT); // Set pin 2 as input
}

void loop() {
  int sensorValue = analogRead(tempPin); // Read analog value from the temperature sensor
  float temperatureC = (sensorValue / 1024.0) * 5.0; // Convert analog value to Celsius
  temperatureC = (temperatureC - 0.5) * 100.0;

  int switchState = digitalRead(2); // Read state of the switch

  if (switchState == HIGH) { // If switch is pressed, print temperature in Fahrenheit
    float temperatureF = (temperatureC * 9) / 5 + 32; // Convert Celsius temperature to Fahrenheit
    Serial.print("Fahrenheit: ");
    Serial.print(temperatureF);
    Serial.println(" F");
  } else { // If switch is not pressed, print temperature in Celsius
    Serial.print("Temperature: ");
    Serial.print(temperatureC);
    Serial.println(" C");
  }

  delay(1000); // Delay of 1 second between each loop
}

```

Code - Centigrade to Serial port, but when button Pressed Fahrenheit Displayed Instead

Screen Shot of Serial Port

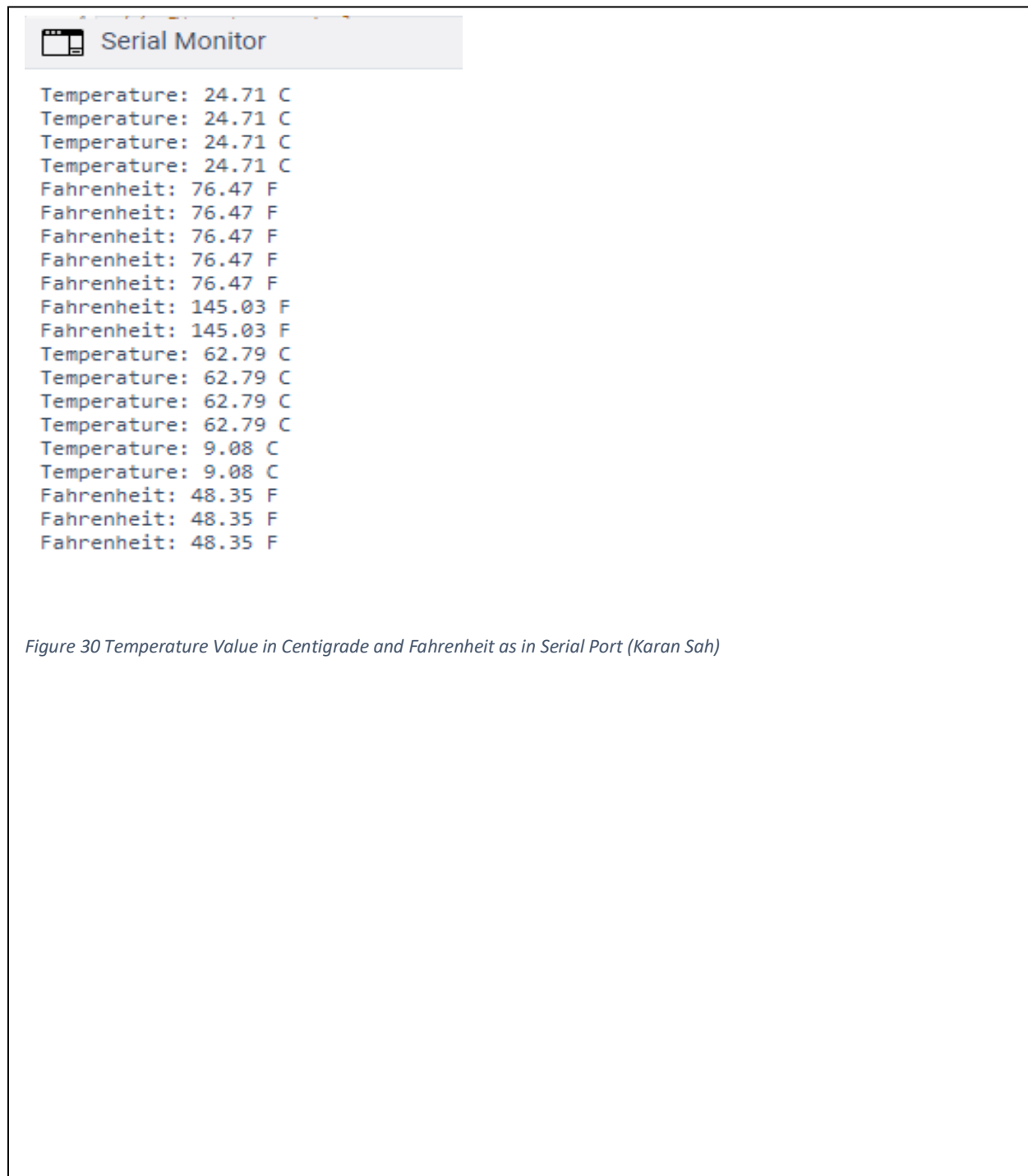


Figure 30 Temperature Value in Centigrade and Fahrenheit as in Serial Port (Karan Sah)

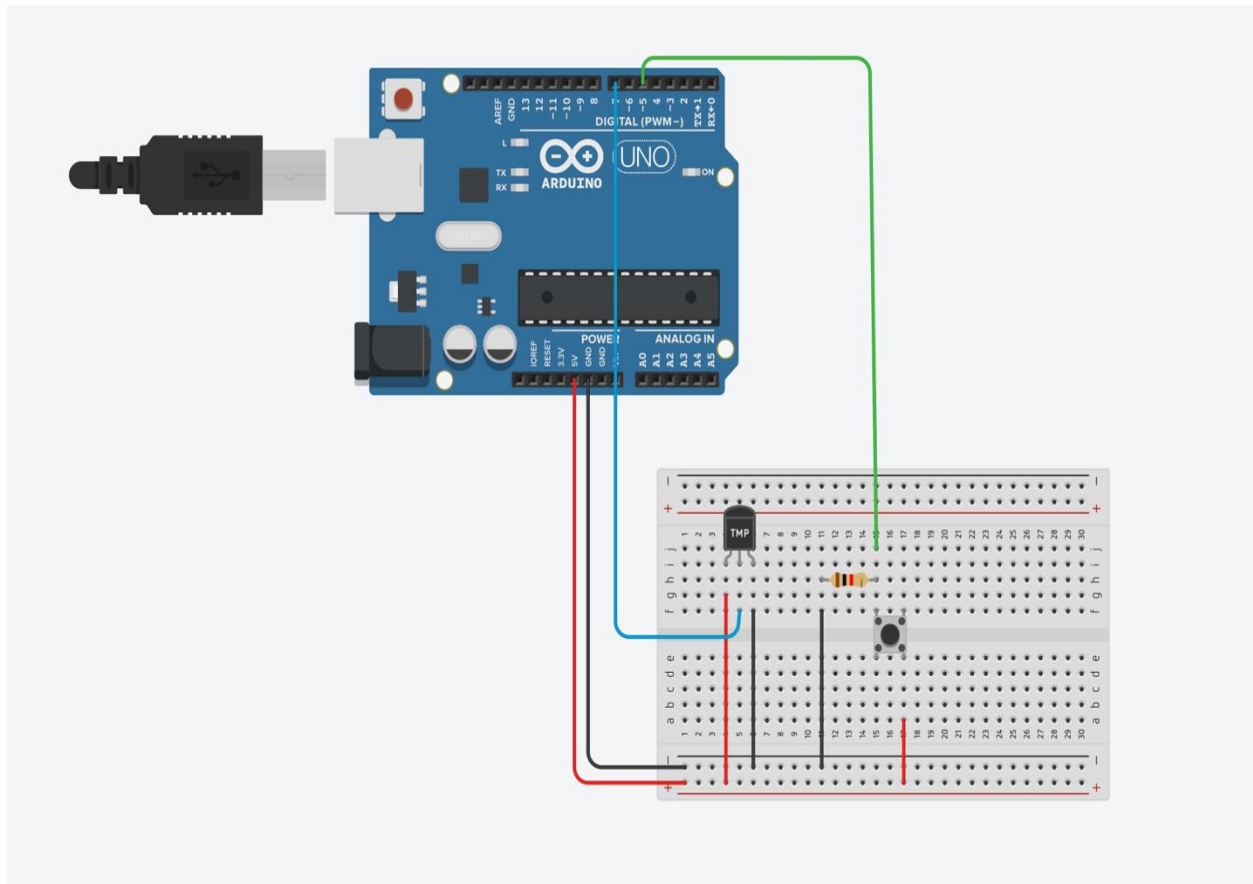


Figure 31 Temperature Sensor (Karan Sah)

Workbook 5

Activity 5.1: RGB Led and switches

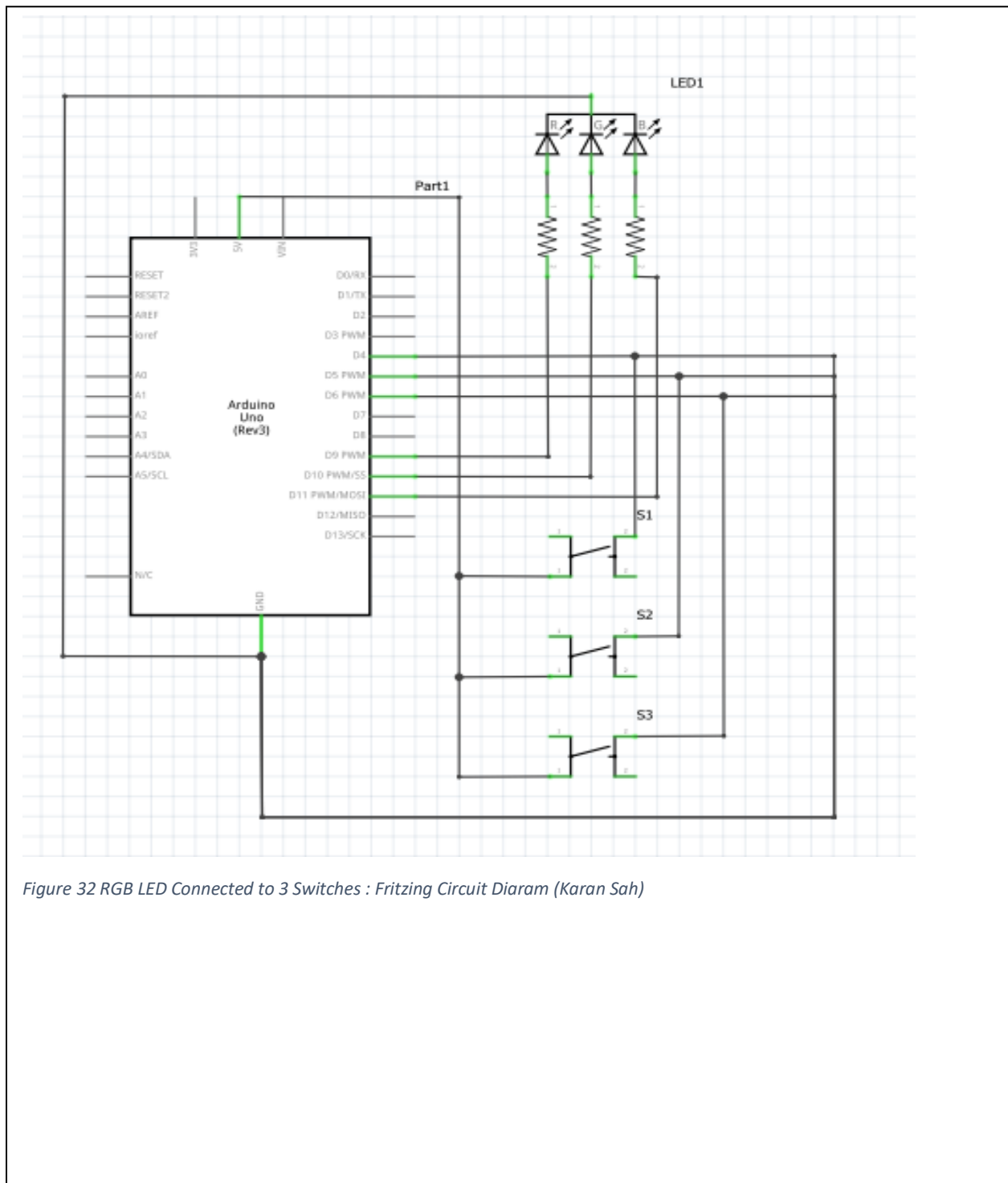


Figure 32 RGB LED Connected to 3 Switches : Fritzing Circuit Diagram (Karan Sah)

Fritzing

Arduino Program

//Karan Sah

```
//2329860

// Set pin 4, 5, 6 as input pins and pin 9, 10, 11 as output pins

void setup()
{
  pinMode(4, INPUT);
  pinMode(5, INPUT);
  pinMode(6, INPUT);
  pinMode(9, OUTPUT);
  pinMode(10, OUTPUT);
  pinMode(11, OUTPUT);
}

void loop()
{
  // Read the value of three buttons and store them in variables a, b, and c
  int a = digitalRead(4);
  int b = digitalRead(5);
  int c = digitalRead(6);

  // If button 1 is pressed, set pin 9 to maximum brightness (255) and the rest to minimum brightness (0)
  if (a == HIGH) {
    analogWrite(9, 255);
    analogWrite(10, 0);
    analogWrite(11, 0);
    delay(1000); // RGB lights on for 1 second
  }

  // If button 2 is pressed, set pin 10 to maximum brightness (255) and the rest to minimum brightness (0)
  else if (b == HIGH) {
```

```

    analogWrite(10, 255);
    analogWrite(9, 0);
    analogWrite(11, 0);
    delay(1000);
}

// If button 3 is pressed, set pin 11 to maximum brightness (255) and the rest to minimum brightness
(0)
else if (c == HIGH) {
    analogWrite(11, 255);
    analogWrite(9, 0);
    analogWrite(10, 0);
    delay(1000);
}

// Turn off all LEDs after each interval
analogWrite(11, 0);
analogWrite(9, 0);
analogWrite(10, 0);
}

```

Activity 5.2: Distance Sensor

Arduino Code

```

//Karan Sah
//2329860

// Define pin numbers for the ultrasonic sensor
int trigPin = 9;
int echoPin = 11;

```

```
// Define variables for storing the duration and distance
long duration;
int distance;

void setup() {
  pinMode(trigPin, OUTPUT); // Set trigPin as an output
  pinMode(echoPin, INPUT); // Set echoPin as an input
  Serial.begin(9600);      // Start Serial communication
}

void loop() {
  // Clear the trigPin
  digitalWrite(trigPin, LOW);
  delayMicroseconds(2);

  // Set the trigPin to HIGH state for 10 microseconds
  digitalWrite(trigPin, HIGH);
  delayMicroseconds(10);
  digitalWrite(trigPin, LOW);

  // Read the echoPin and return the sound wave travel time in microseconds
  duration = pulseIn(echoPin, HIGH);

  // Calculate the distance in centimeters
  distance = duration * 0.034 / 2;

  // Print the distance on the Serial monitor
  Serial.print("Distance: ");
  Serial.print(distance);
```

```
Serial.println(" cm");
```

```
// Add a delay of 2 seconds
```

```
delay(2000);
```

```
}
```

Take a picture of your distance sensor and include it here, please reduce the size and quality as it will be too large else 😊

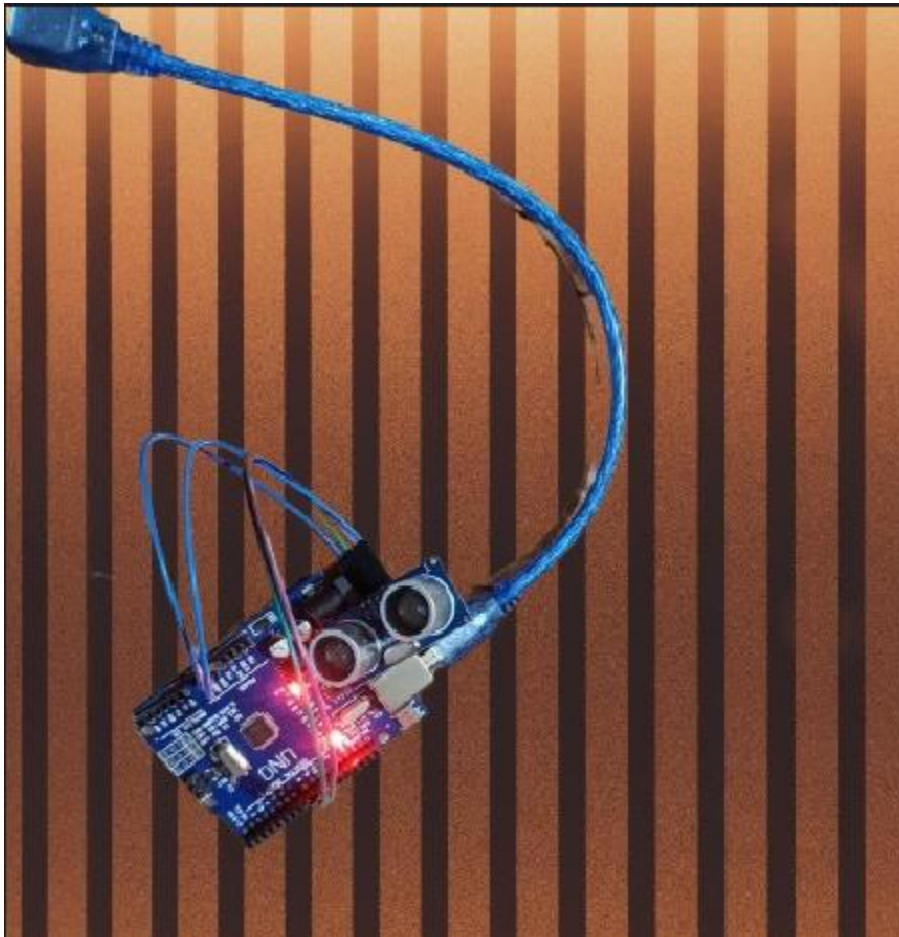


Figure 33 Ultrasonic Distance Sensor : Arduino Setup (Karan Sah)



```
Distance: 10 cm
Distance: 2 cm
Distance: 211 cm
Distance: 11 cm
Distance: 128 cm
Distance: 2180 cm
Distance: 2181 cm
Distance: 118 cm
Distance: 63 cm
```

*Figure 34*Output of Ultrasonic Distance Sensor : Serial Port (Karan Sah)

Activity 5.3: 1602 LCD Display

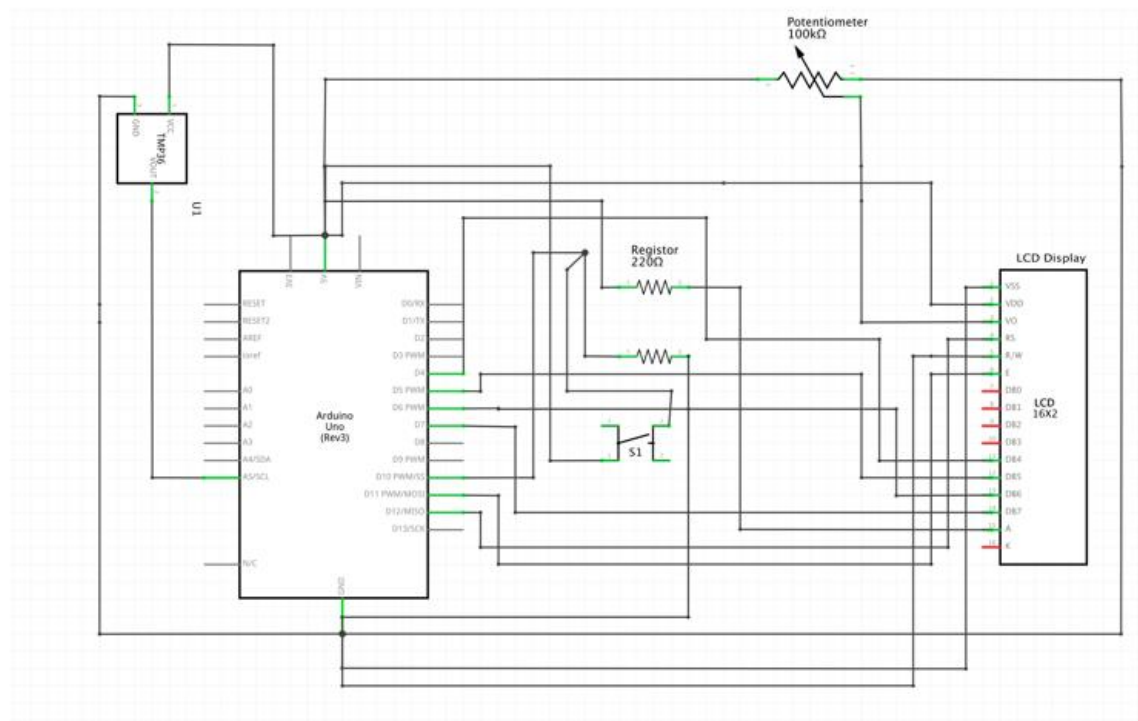


Figure 35 LCD Display Connected to DHT11 Temperature Sensor : Fritzing Circuit Diagram (Karan Sah)

Fritzing

//Karan Sah

//2329860

```
#include <LiquidCrystal.h>

// Define pin numbers for components
const int SWITCH_PIN = 10;
const int RS_PIN = 12;
const int EN_PIN = 11;
const int D4_PIN = 4;
const int D5_PIN = 5;
const int D6_PIN = 6;
const int D7_PIN = 7;

// Creating LiquidCrystal object using defined components
LiquidCrystal lcd(RS_PIN, EN_PIN, D4_PIN, D5_PIN, D6_PIN, D7_PIN);

void setup()
{
    pinMode(SWITCH_PIN, INPUT); // Set switch pin mode

    lcd.begin(16, 2); // Initialize LCD object

    lcd.setCursor(0, 0); // Set LCD cursor to position (0, 0) and print text
    lcd.print("Karan010");
}

void loop()
{
    int switchState = digitalRead(SWITCH_PIN); // Read switch state
    int sensorValue = analogRead(A5); // Read analog sensor value
```

```
float degreeC = (sensorValue * 5.0 / 1024.0 - 0.5) * 100.0; // Convert sensor value to temperature in Celsius
```

```
float degreeF = degreeC * 1.8 + 32; // Convert Celsius temperature to Fahrenheit
```

```
if (switchState == LOW) {
```

```
    lcd.setCursor(0, 1); // If switch is pressed, display temperature in Celsius on LCD
```

```
    lcd.print("Temp: ");
```

```
    lcd.print(degreeC);
```

```
    lcd.print(" °C");
```

```
} else {
```

```
    lcd.setCursor(0, 1); // If switch is not pressed, display temperature in Fahrenheit on LCD
```

```
    lcd.print("Temp: ");
```

```
    lcd.print(degreeF);
```

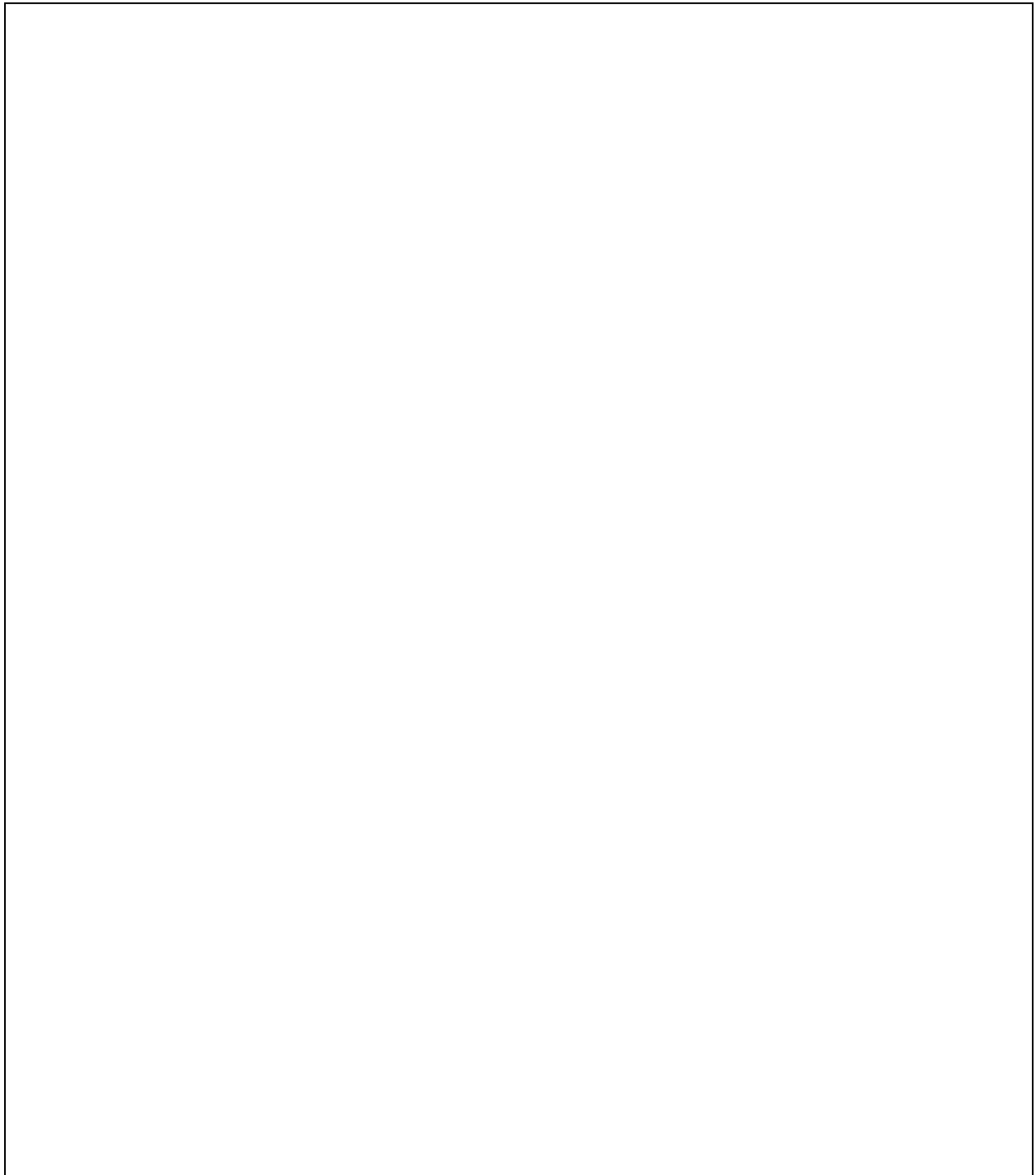
```
    lcd.print(" °F");
```

```
}
```

```
}
```

Arduino Program

Take a picture of your LCD and include it here, please reduce the size and quality as it will be too large



else 😊

Workbook 6

Activity 6.1: PWM

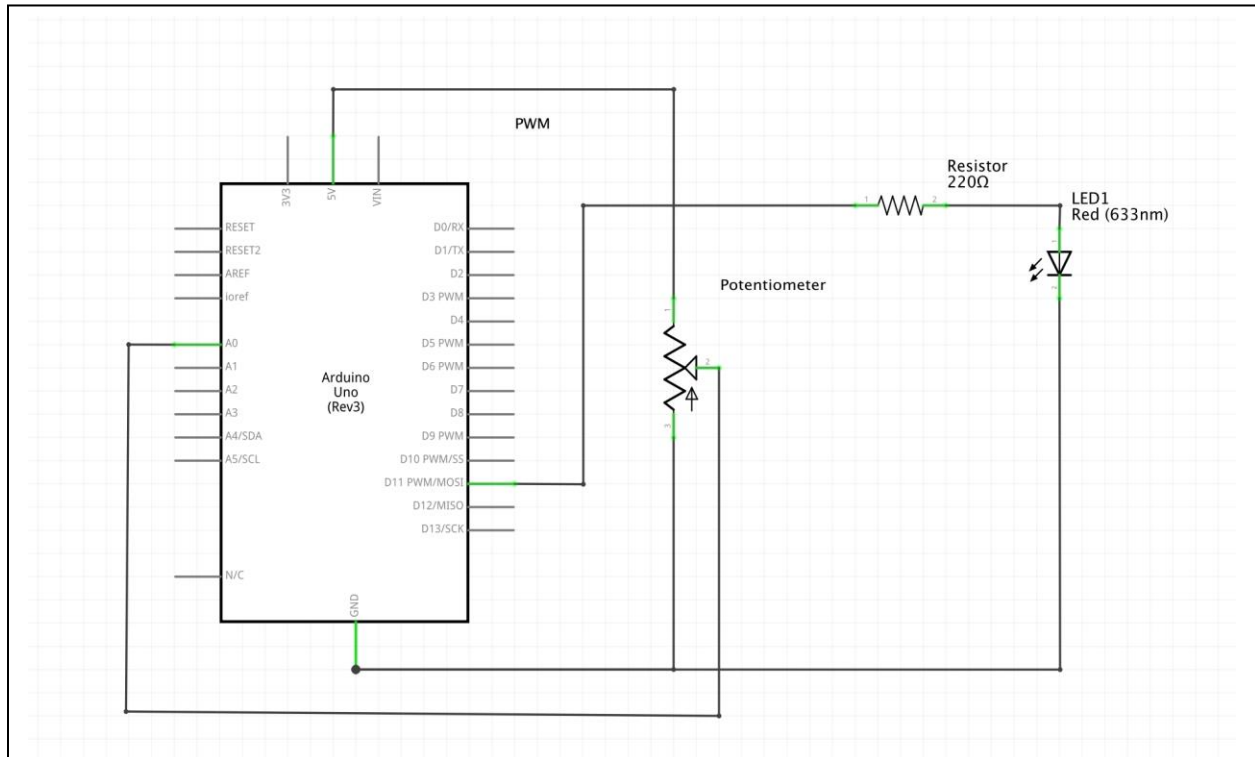


Figure 36 LED Light Controlled as Shown by Diagram by Potentiometer (Karan Sah)

```
//Karan Sah
//2329860
// Program: Control of LED using a potentiometer
// Variable for storing analog values
int pot;
// Potentiometer connected to pin A0
// LED connected to pin 11
int ledPin = 11;

void setup()
{
  Serial.begin(9600);
  // Initialise the analog pin A0 as input
  pinMode(A0, INPUT);
  // Initialise pin 11 as output
  pinMode(ledPin, OUTPUT);
}

void loop()
{
  // Read the analog value and store it in a variable
  pot = analogRead(A0);
  // Map the variable from 0-1023 to 0-255
  pot = map(pot, 0, 1023, 0, 255);
  // Control the brightness of the LED
  analogWrite(ledPin, pot);
  Serial.println(pot); // Print the potentiometer value
  // Delay of 200ms
  delay(200);
}
```

```
}
```

Arduino Program

Workbook 7

Activity 7.1: 2 Arduinos – using Digital Pins

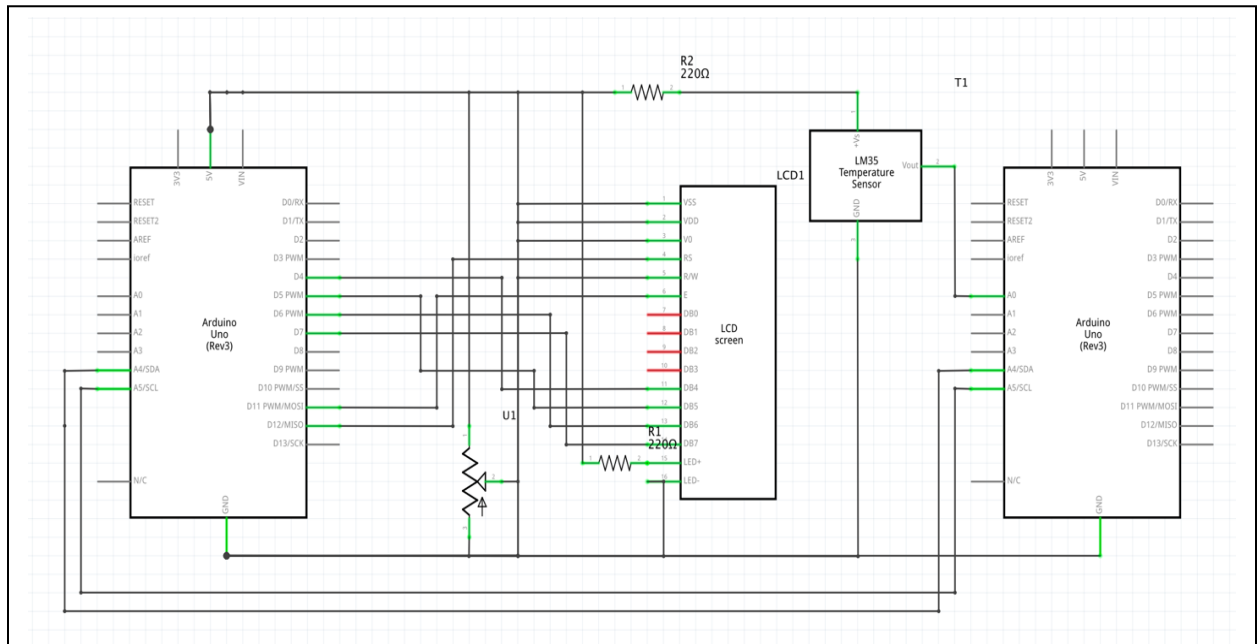


Figure 37 LCD Scree Displaying Temperature by Communication Two Arduinos


```
//Karan Sah
//2329860
// Code for LCD Display
// Code for Receiver Arduino

#include <LiquidCrystal.h>
#include <Wire.h>

LiquidCrystal lcd(12, 11, 7, 6, 5, 4); // Initializing LCD display

void setup()
{
  lcd.begin(16, 2);    // Start LCD display
  Serial.begin(9600);
  Wire.begin(8);      // Begin I2C or TWI with address 8 (slave address)
  /* Act as a slave component (dataReceiver) to display temperature reading
  when the master component (dataSender) wants to write temperature data to it */
  Wire.onReceive(dataReceived);
  lcd.print("Waiting for data...");
  lcd.setCursor(0,1);
  lcd.print("Please wait...");
  pinMode(8, OUTPUT);
}

void loop()
{
  // Add your code here
}
```

```
void dataReceived(int x) // x indicates how many times data has been received
{
    int CelsiusVal = Wire.read(); // Reading the sent value
    Serial.print("Temperature in Celsius: ");
    Serial.print(CelsiusVal);
    Serial.print("degC");
    Serial.println();

    int FahrenheitVal = Wire.read(); // Reading the sent value
    Serial.print("Temperature in Fahrenheit: ");
    Serial.print(FahrenheitVal);
    Serial.print("degF");
    Serial.println();

    lcd.clear();

    lcd.setCursor(0, 0);
    lcd.print("Celsius: ");
    lcd.print(CelsiusVal);
    lcd.print("C");

    lcd.setCursor(0, 1);
    lcd.print("Fahrenheit: ");
    lcd.print(FahrenheitVal);
    lcd.print("F");
}
```

```

//Karan Sah

//2329860

// Code for Temperature Sensor Connected Arduino
// Allows communication with I2C/TWI devices
#include <Wire.h>

int tempPin = A0; // Initializing tempPin at A0

void setup()
{
    delay(500); // Wait for 500 milliseconds
    Wire.begin(); // Begin I2C communication for master or sender
    Serial.begin(9600);
}

void loop()
{
    float tempData = analogRead(tempPin); // Read ADC value from the temperature sensor
    float Vout = (tempData * 5) / 1023.0; // Convert ADC value to voltage (assuming 5V reference voltage)
    int celsius = (Vout - 0.5) * 100; // Convert the voltage into Celsius
    int fahrenheit = celsius * (9.0 / 5.0) + 32.0; // Convert the data into Fahrenheit

    Wire.beginTransmission(8); // Transmit to device with address 8
    Wire.write(celsius); // Send temperature value in Celsius
    Wire.write(fahrenheit); // Send temperature value in Fahrenheit
    Wire.endTransmission(); // Stop transmitting
    delay(1000);
}

```

Arduino Program

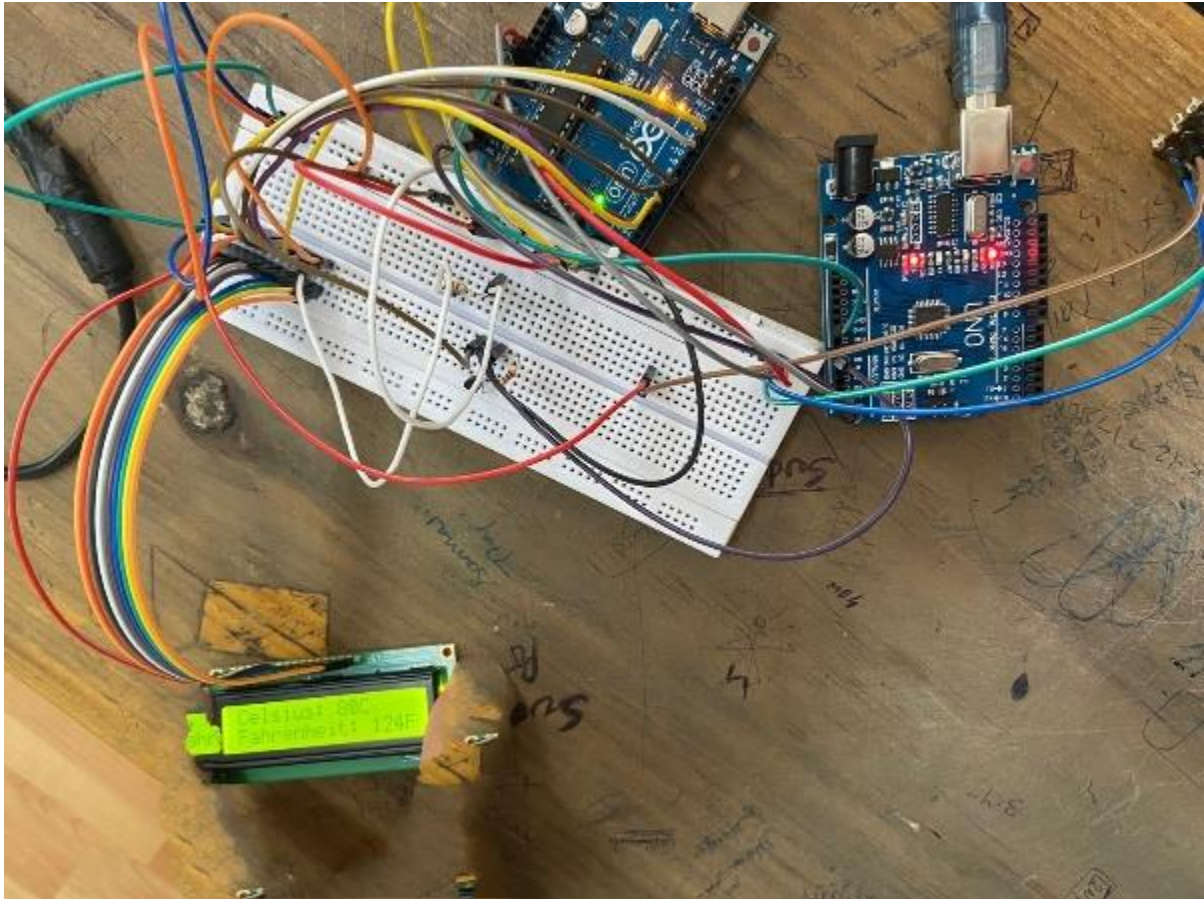


Figure 38 LCD Screen Displaying Temperature Measured Using Two Arduinos (Karan Sah)

Activity 7.2: 2 Arduinos – using Serial I/O

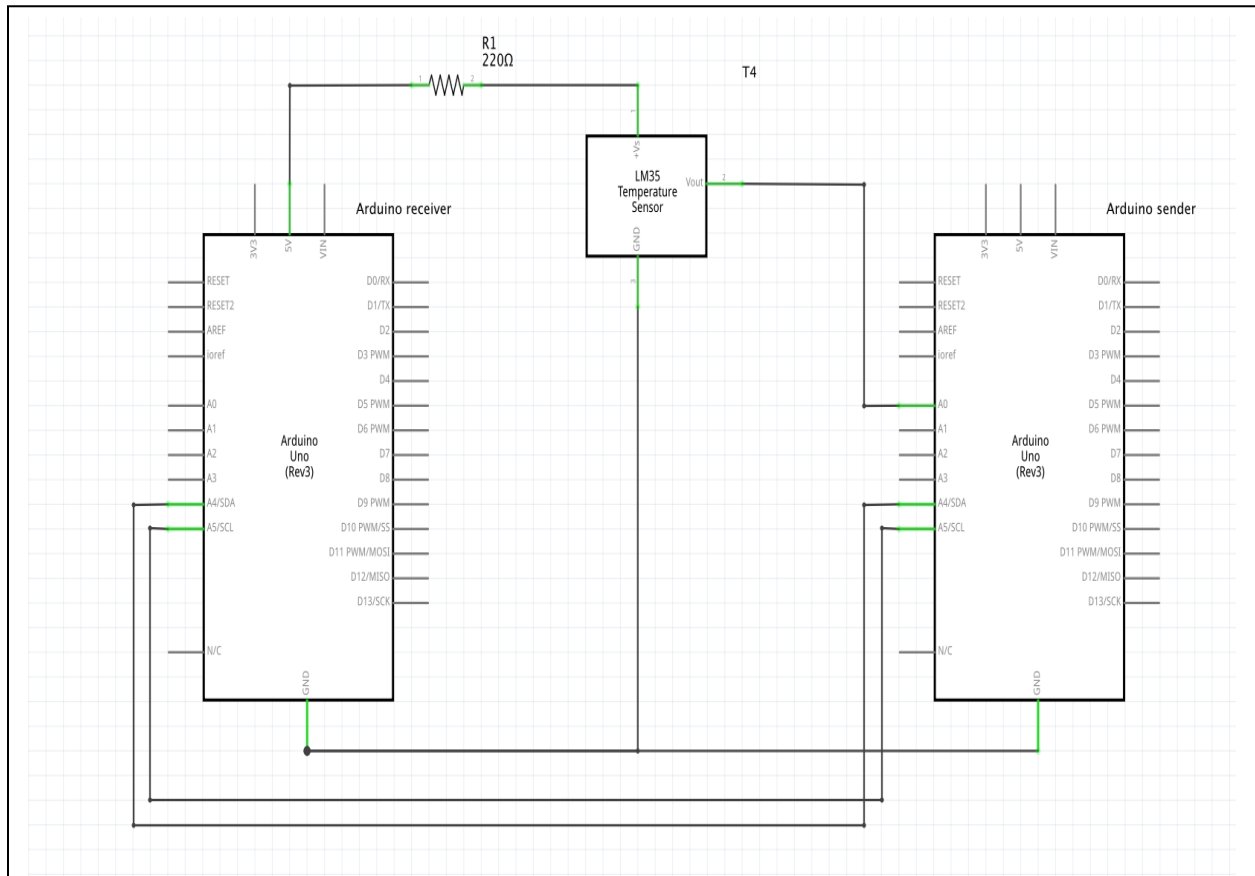


Figure 39Serial Monitor Displaying Measured Temperatur Using Two Arduinos (Karan Sah)

Fritzing

//Karan Sah

//2329860

```

// Code for Receiver Arduino

#include <Wire.h>

void setup() {
  Serial.begin(9600);
  Wire.begin(8); // Set I2C address to 8 (slave address)
  Wire.onReceive(receiveEvent); // Set receiveEvent as the function that gets called when data is
  received
  Serial.println("Waiting for data...");
}

void loop() {
  // The loop function is empty as all the functionality is handled in the receiveEvent function
}

void receiveEvent(int bytesReceived) {
  if (bytesReceived == 2) { // Check if two bytes were received
    int celsius = Wire.read(); // Read the Celsius value
    int fahrenheit = Wire.read(); // Read the Fahrenheit value

    Serial.print("Temperature in Celsius: ");
    Serial.print(celsius);
    Serial.print(" degC, ");
    Serial.print("Temperature in Fahrenheit: ");
    Serial.print(fahrenheit);
    Serial.print(" degF");
    Serial.println();
  }
}

```

```
// Code for Sender Arduino
```

```
#include <Wire.h>
```

```
int tempPin = A0;
```

```
void setup() {
```

```
    delay(500);
```

```
    Wire.begin(); // Start I2C communication
```

```
    Serial.begin(9600);
```

```
}
```

```
void loop() {
```

```
    float tempdata = analogRead(tempPin);
```

```
    float Vout = (tempdata * 5) / 1023.0;
```

```
    int celsius = (Vout - 0.5) * 100;
```

```
    int fahrenheit = celsius * 1.8 + 32;
```

```
    Wire.beginTransmission(8); // Set the receiver's I2C address to 8
```

```
    Wire.write(celsius); // Write the Celsius value
```

```
    Wire.write(fahrenheit); // Write the Fahrenheit value
```

```
    Wire.endTransmission();
```

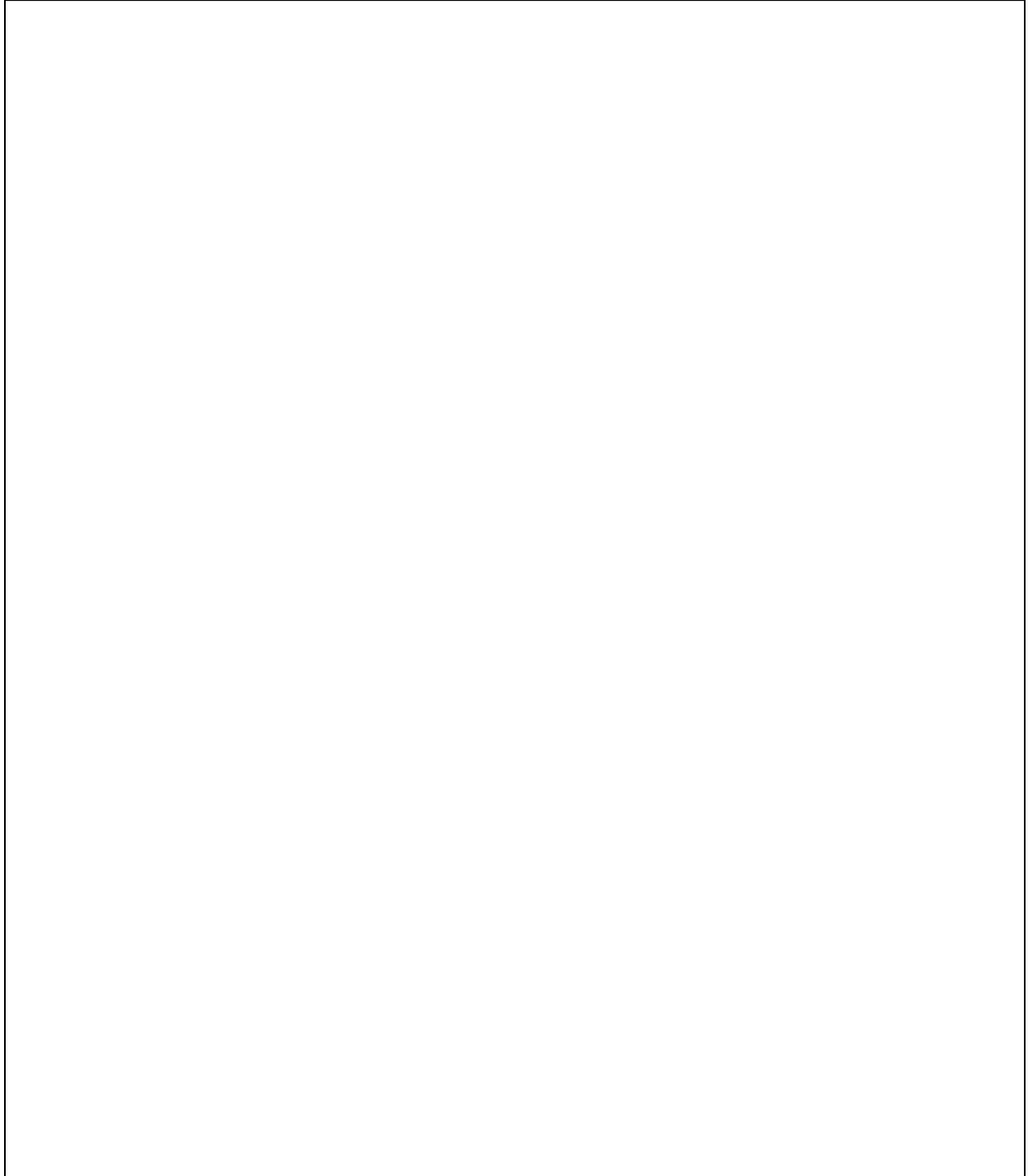
```
    delay(1000);
```

```
}
```

Arduino Program

Workbook 8

Activity 8.1: Stepper Motor Circuit Diagram



Circuit Diagram


```

// Arduino Program

// Importing AccelStepper motor library
#include <AccelStepper.h>
#define HALFSTEP 8

// Motor pin definitions
#define motorPin1 8 // IN1 on the ULN2003 driver 1
#define motorPin2 9 // IN2 on the ULN2003 driver 1
#define motorPin3 10 // IN3 on the ULN2003 driver 1
#define motorPin4 11 // IN4 on the ULN2003 driver 1

// Initialize with pin sequence IN1-IN3-IN2-IN4 for using the AccelStepper with 28BYJ-48
AccelStepper stepper1(HALFSTEP, motorPin1, motorPin3, motorPin2, motorPin4);

void setup() {
    stepper1.setMaxSpeed(2000.0);
    stepper1.setAcceleration(2000.0);
    stepper1.setSpeed(2000);
    stepper1.moveTo(2000);
}

void loop() {
    // Change direction when the stepper reaches the target position
    if (stepper1.distanceToGo() == 0) {
        stepper1.moveTo(stepper1.currentPosition());
    }
}

// End of program

```

Activity 8.2: 2 Stepper Motors

```
#include <Stepper.h>
#include <AccelStepper.h>

#define HALFSTEP 8

int Pin1 = 8; // IN1 is connected to pin 8
int Pin2 = 9; // IN2 is connected to pin 9
int Pin3 = 10; // IN3 is connected to pin 10
int Pin4 = 11; // IN4 is connected to pin 11

int switchCW = 2;
int switchCCW = 3;

int pole1[] = {0, 0, 0, 0, 0, 1, 1, 1, 0};
int pole2[] = {0, 0, 0, 1, 1, 1, 0, 0, 0};
int pole3[] = {0, 1, 1, 1, 0, 0, 0, 0, 0};
int pole4[] = {1, 1, 0, 0, 0, 0, 0, 1, 0};

int poleStep = 0;
int dirStatus = 3;

void setup() {
    // Code setup goes here
}
```

Arduino Program

Workbook 9

Activity 9.1: Windscreen Wiper Code using Servos & Temperature Sensor

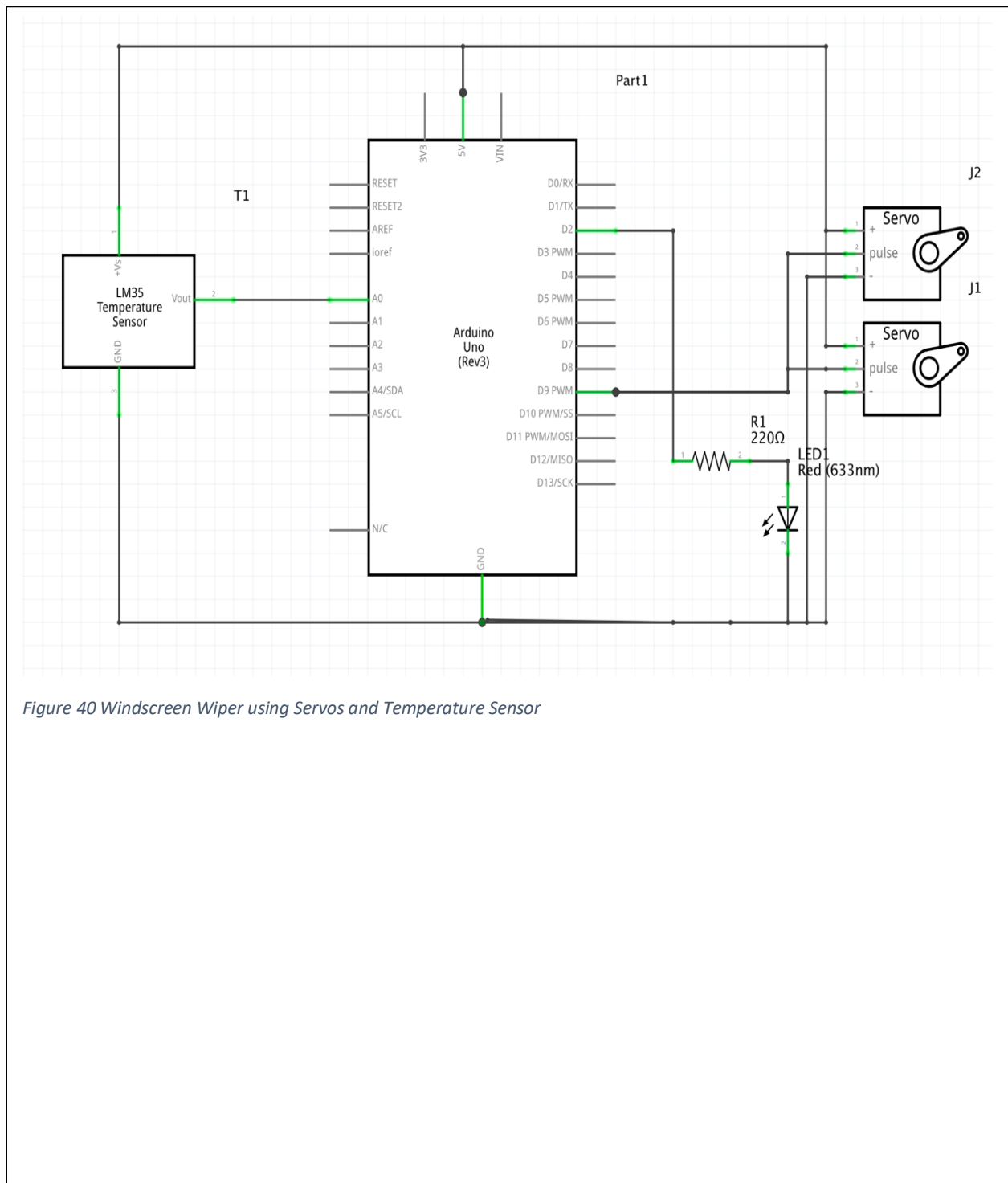


Figure 40 Windscreen Wiper using Servos and Temperature Sensor

```

// Code for controlling two servos based on temperature reading
// Author: Karan Sah
// Student ID: 2329860

#include <Servo.h>

Servo myServo1;      // Create a servo object for servo 1
Servo myServo2;      // Create a servo object for servo 2
int servoPin1 = 9;    // Pin number for servo 1
int servoPin2 = 10;   // Pin number for servo 2
int angle = 0;        // Variable to store servo angle
int tempPin = A0;     // Pin number for temperature sensor

void setup() {
    myServo1.attach(servoPin1); // Attaching servo 1 to its pin
    myServo2.attach(servoPin2); // Attaching servo 2 to its pin
    pinMode(tempPin, INPUT);    // Set temperature pin as input
}

void loop() {
    float tempdata = analogRead(tempPin); // Read temperature sensor value
    float Vout = (tempdata * 5) / 1023.0; // Convert sensor value to voltage
    int temp = (Vout - 0.5) * 100;        // Convert voltage to temperature in Celsius

    if (temp >= 25) {
        // If temperature is greater than or equal to 25 degrees
        // Move servos from 0 to 90 degrees and then back to 0 degrees
        for (angle = 0; angle <= 90; angle++) {
            myServo1.write(angle);

```

```
    myServo2.write(angle);  
    delay(10);  
}  
  
for (angle = 90; angle >= 0; angle--) {  
    myServo1.write(angle);  
    myServo2.write(angle);  
    delay(10);  
}  
} else {  
    // If temperature is below 25 degrees  
    // Set servos to 0 degrees  
    myServo1.write(0);  
    myServo2.write(0);  
    delay(15);  
}  
}
```

Arduino Code

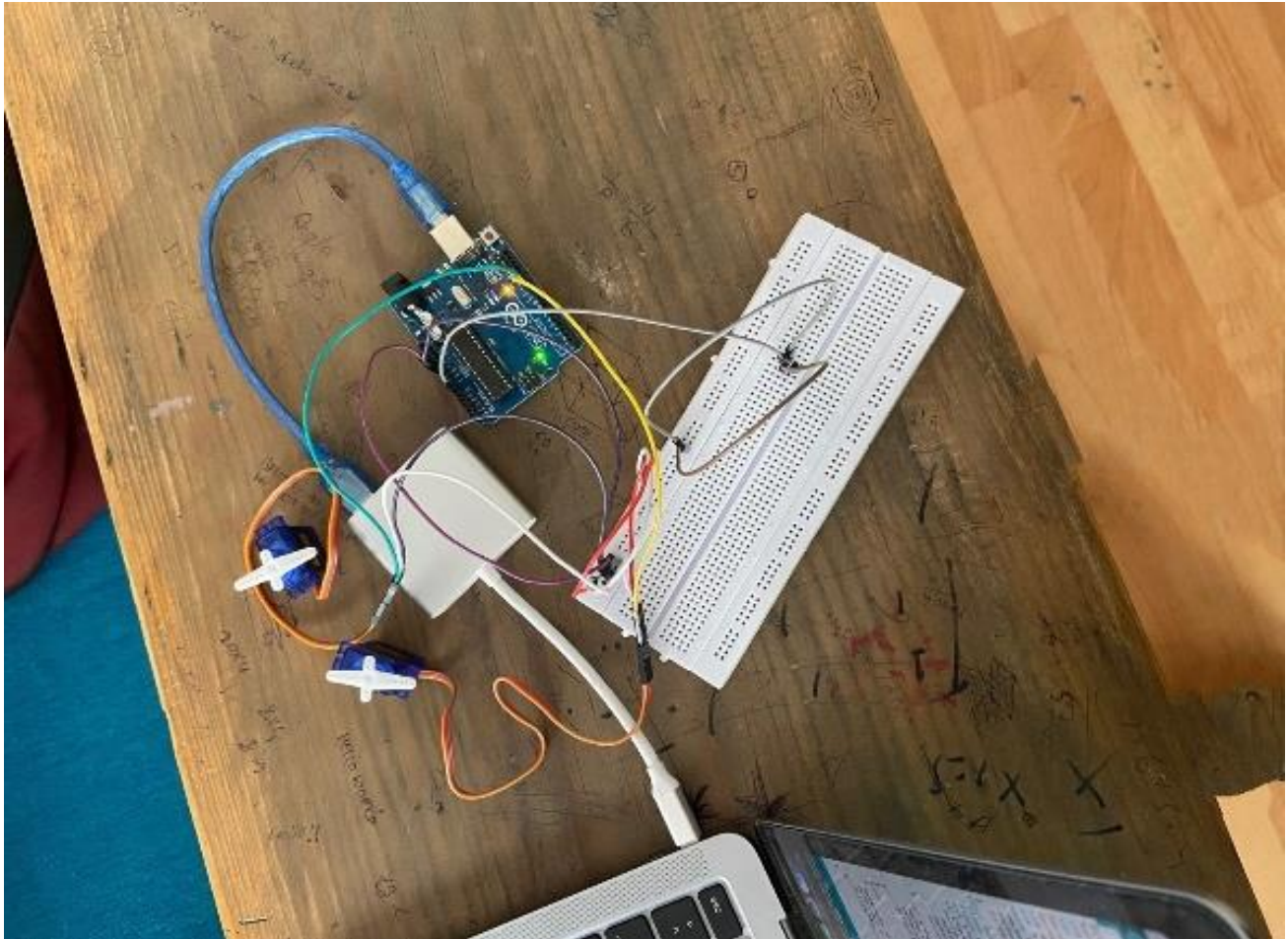


Figure 41 Windscreen Wiper Using Servos and Temperature Sensor : Arduino Program (Karan Sah)

Individual Project (50%)

Rationale

Throughout the module you have used a range of sensors and actuators with an Arduino to complete weekly tasks. For the mini project we would like you to research and create a small embedded project in an area of your choice, such as:

- Games
- Networking
- IT Security
- Systems Engineering
- Smart Technology
- Artificial Intelligence

Previous projects have included a reaction game that gives a score depending on how fast you hit a button, this has buttons to restart the application, and an LCD to show scores, and information.

This project should be your own work, **YOU MUST NOT COPY A PROJECT FROM THE INTERNET.**

Timescales

This project should be started around week 5 and continue until the deadline, when it will be submitted in the Portfolio.

Equipment

You are free to use Tinkercad, or your own kit.

The Project

Step 1 produce a detailed description of your project.

This should clearly describe what you are intending to build and may contain some diagrams of how the sensor/switches input is to be processed by the Arduino. Then what kind of output is intended to be seen or heard by the user. Please mention any tools you intend to use.

Step 2 Circuit Diagram & Fritzing Schematic

You are required to produce a circuit diagram of your work showing any calculations you made, so these might be suitable resistor values for any LED's you use. These calculations are covered on the module. The circuit diagram should not be hand drawn but should follow the format of circuits from the module.

Step 3 A Program

You will need to write some software for this project and a listing of the code with suitable comments will need to be included.

Step 4 Testing

You will be required to produce some suitable test data that you would expect to be able to measure such as voltages, test code.

Once your prototype is complete you will be expected to test your circuit and compare the actual values to your initial test data, and comment on the results.

Step 5 Conclusions

You are required to write a summary of the work along with a short half page reflection on how you found the work.

Layout

The report should be suitably laid out for a report, using headings, references if required in Harvard style, and appendices used for any lengthy code. All diagrams should be produced on a PC, and hand-written work is not acceptable.

Marking

All sections carry equal marks.